# RBSRN: Role-Aware and Background-Conditioned Residual Refinement for Multi-Step Regional Traffic Flow Forecasting


## Abstract

Short-term, multi-step regional traffic flow forecasting uses recent observations to estimate future inflow and outflow for traffic dispatch. When local flow deviates from routine patterns, self-evolution and cross-region historical information play different correction roles. Historical correlations alone do not establish their relevance in the current traffic context or at different forecast horizons. We propose the Role-aware and Background-conditioned Spatiotemporal Residual Network (RBSRN), which refines a base prediction through two parallel branches. The first fuses residual corrections from self-evolution and directed time-lagged experts according to the current state. The second performs Background-conditioned Cross-region Message Selection (BCMS), jointly selecting neighbor–lag–scale messages for each forecast step using source and target traffic backgrounds. We evaluated RBSRN against 13 baseline methods on four inflow and outflow forecasting tasks in Chengdu and Shanghai. RBSRN achieved the lowest mean absolute error (MAE) averaged over 12 forecast steps under positive-target evaluation. Its MAE was 0.43%–2.00% lower than that of the strongest baseline method for each task. Fixed-model residual-removal diagnostics showed that, among high-deviation samples, the MAE benefit of self-evolution correction at the 30-minute horizon was 1.15–2.82 times as large for rapidly changing histories as for slowly changing histories. BCMS, in turn, provided its largest mean benefit in the group with the lowest history-derived background-discrepancy proxy values at all 12 forecast steps. Together, these results support prediction correction conditioned on traffic state and background.



**Keywords:** regional traffic flow forecasting; multi-step forecasting; spatiotemporal residual learning; role-aware gating; background-conditioned historical message selection


# 1 Introduction

Anticipating travel demand across urban regions supports fleet dispatch and the allocation of transport resources [1]. Regional inflow and outflow describe trips arriving in and departing from each region within a given interval [2]; their variation determines where demand concentrates, how large it becomes, and how long it persists. Dispatch decisions require forecasts of successive increases, decreases, and spatial shifts in demand to guide vehicle deployment and regional service provision [3], [4]. Short-term multi-step traffic flow forecasting therefore jointly estimates regional inflow and outflow at successive future time steps from recent observations, supporting dispatch decisions over the next 5–60 minutes. Traffic event information has been used to supplement historical traffic observations and improve short-term flow forecasting [5]. Improving multi-step regional forecasting requires models to capture routine spatiotemporal patterns while responding to local changes in demand.

Traffic flow forecasting has progressed through advances in spatial dependency learning, temporal pattern representation, and multiscale dynamics modeling. For regional flow forecasting, ST-ResNet uses residual convolutions to jointly model temporal closeness, periodicity, and trends. In its single-step TaxiBJ experiment at a 30-minute resolution, the best configuration

reduced RMSE from DeepST's 18.18 to 16.69, a relative decrease of approximately 8.2% [6]. Spatiotemporal graph networks combine inter-node relations with temporal encoding, learning spatial dependencies through diffusion convolution, graph convolution, and adaptive adjacency matrices [7], [8], [9]. STWave and TEDDCF use frequency decomposition and two-branch encoding to represent low-frequency trends and high-frequency dynamics separately [10], [11]. ST-GDN further combines local geographic dependencies, global regional associations, and multiscale temporal attention to organize historical information [12]. These studies broaden the historical information available to forecasting models, drawing on node trajectories, cross-region relations, and temporal patterns at different scales. When local flow deviates from routine patterns, however, the relevance of this historical evidence needs to be reassessed in light of the current state. Determining its relevance at different forecast horizons and translating it into effective prediction corrections remain important questions for short-term multi-step forecasting.

These issues concern both the sources of evidence and the conditions under which historical messages are useful. A node's own history directly records the continuation and turning points of its local state, whereas observations from other regions provide cross-region references through directed time-lagged associations [13]. These sources differ in their spatial origins, temporal alignment, and correction roles. Even for the same neighbor, historical messages correspond to different lag positions and scales of variation; their relevance must be assessed using the source and target traffic backgrounds and the forecast horizon. Figure 1 illustrates these differences using outflow from grid cell 601 in Chengdu in November 2016: for two recent traffic states, the historical correlations between the target and a lag-aligned neighboring region were 0.859 and 0.907, respectively, whereas the mean absolute differences between their observed trajectories over the subsequent 30–60 minutes were 0.5 and 6.0 counts per 15 min. Both states exhibited strong historical associations but differed in how far their subsequent trajectories diverged. This observation focuses attention on the conditions under which historical information remains useful. Self-evolution and cross-region time-lagged evidence need distinct roles in correcting the base prediction, while historical messages need to be selected for each future forecast step according to the current background.

Existing approaches address heterogeneous dynamics and cross-region information through signal disentanglement, expert routing, and delay awareness. D2STGNN separates diffusion and inherent signals through gating and residual decomposition, then models them with different modules [14]. TESTAM uses temporal, static-graph, and dynamic-graph experts, adapting to different traffic conditions through routing [15]. PDFormer combines dynamic spatial attention with delay-aware feature transformations to address temporal misalignment in cross-region information [16]. These studies provide a basis for distinguishing sources of dynamics and establishing spatiotemporal associations, making the organization of prediction evidence more explicit. For multi-step regional forecasting under local deviations, linking such differentiated modeling to correction benefits requires understanding when self-evolution and directed time-lagged information improve the base prediction, and how the benefits of

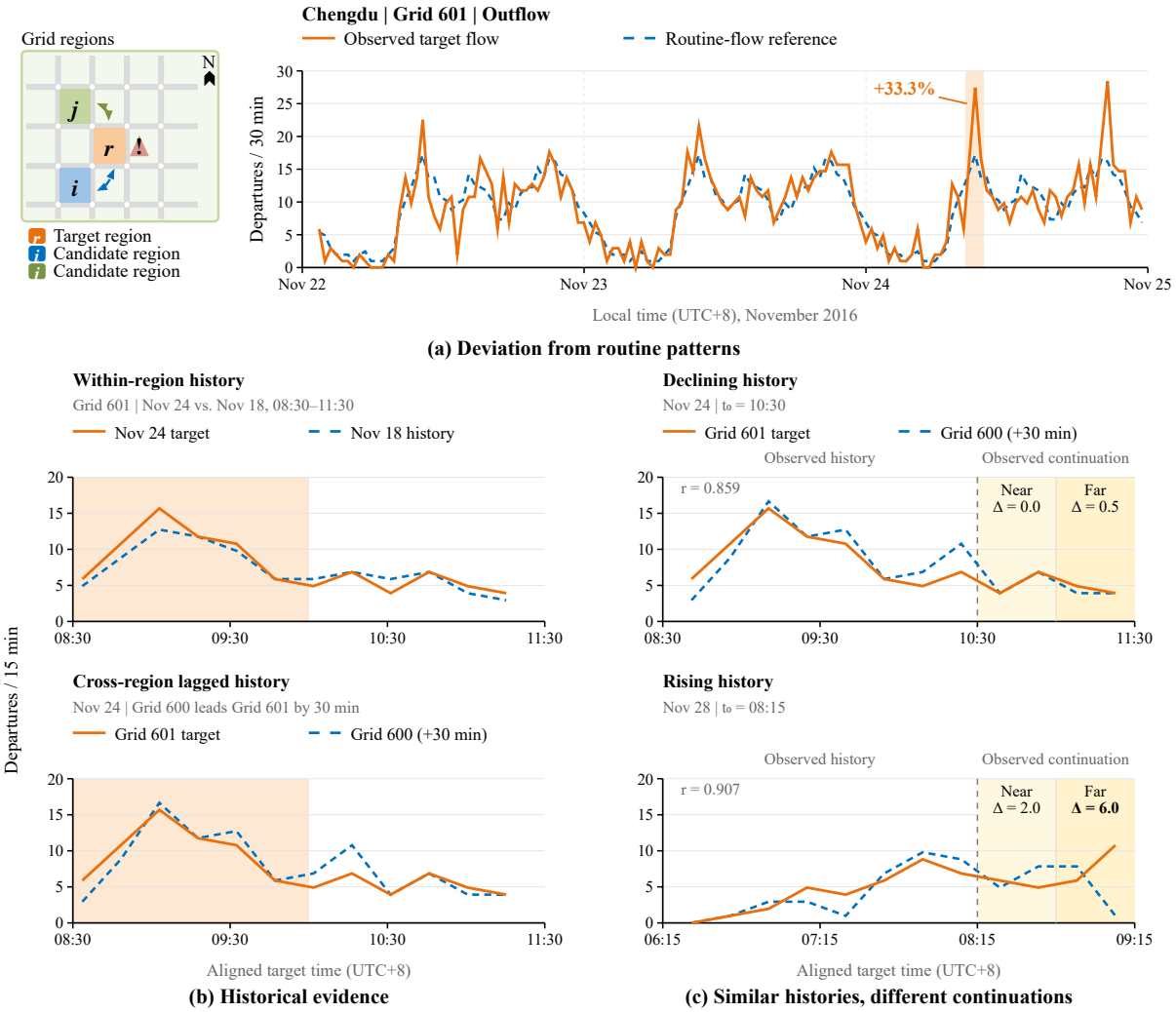



**Figure 1** Observed deviations from routine flow patterns and historical references. (a) Routine-flow reference and observed flow. (b) Within-region and cross-region historical evidence. (c) Divergent subsequent trajectories following similar histories.

cross-region historical messages vary with source–target background discrepancy and forecast horizon.

We introduce the Role-aware and Background-conditioned Spatiotemporal Residual Network (RBSRN) to organize correction evidence and select historical messages when local flow deviates from routine patterns. RBSRN refines a base prediction through adaptive fusion of heterogeneous experts and background-conditioned selection of historical messages. Shared dual-frequency spatiotemporal encoding first extracts low- and high-frequency representations and generates the base prediction. The State-Evidence Expert Residual Refinement Module (SERR) assigns distinct roles to self-evolution and directed time-lagged experts and allocates their correction contributions according to the current state. Background-conditioned Cross-region Message Selection (BCMS) uses the low-frequency source and target traffic backgrounds to jointly select high-frequency historical messages for each forecast step. Both modules read the shared representations in parallel, and their corrections are added at the prediction stage to jointly forecast regional inflow and outflow over multiple future steps.

The main contributions are as follows:

1. To account for heterogeneous prediction evidence and allocate its contributions under deviations from routine patterns, we introduce SERR. Its state-conditioned gate combines residual corrections from self-evolution and directed time-lagged experts to refine the base prediction according to the current state. Ablation experiments on Chengdu showed that removing either expert increased both MAE and RMSE for inflow and outflow.

2. To account for changes in the relevance of cross-region historical evidence across traffic backgrounds and forecast horizons, we introduce BCMS. It jointly selects neighbor–lag–scale messages for each forecast step, providing background-conditioned and horizon-specific residual corrections to the base prediction. Fixed-model residual-removal diagnostics in both cities showed larger average MAE benefits from these corrections when the history-derived proxy for background discrepancy between the target and candidate source regions was smaller.

3. We systematically evaluated RBSRN's forecasting performance and residual refinement on the Chengdu and Shanghai datasets. In the overall comparison with 13 representative baseline methods, RBSRN achieved the lowest AVG12 MAE on all four city–channel forecasting tasks, reducing MAE by 0.43%–2.00% relative to each task's strongest baseline method. It also achieved the lowest AVG12 RMSE on all four tasks.

# 2 Related Work

## 2.1 Spatiotemporal Representations and Temporal Pattern Disentanglement

Traffic flow forecasting requires models to capture complex spatial dependencies and temporal dynamics. Graph neural networks have become an important approach to traffic forecasting by explicitly modeling associations between road nodes [17]. DCRNN models traffic state propagation as diffusion on a directed graph and combines diffusion convolution with recurrent neural networks to capture spatial propagation and temporal evolution [7]. STGCN combines graph convolution with temporal convolution for efficient spatiotemporal feature extraction [8]. Graph WaveNet learns latent spatial relations through an adaptive adjacency matrix and expands the temporal receptive field through dilated convolutions, extending modeling beyond a predefined topology [9]. STID uses spatial and temporal identity embeddings to distinguish samples from different nodes and time slots [18]. These methods establish a basic paradigm that combines spatial relation modeling with time-series encoding for traffic forecasting.

Subsequent studies have enhanced spatiotemporal representations through attention, adaptive relation learning, and joint spatiotemporal graph construction. ASTGCN and GMAN use spatial and temporal attention to model spatial correlations and temporal dependencies, focusing on relevant information at different locations and times [19], [20]. MTGNN and AGCRN learn node relations from data, reducing reliance on manually constructed adjacency matrices [21], [22]. STSGCN synchronously models inter-node and inter-time associations in local spatiotemporal graphs [23]. STFGNN fuses spatial adjacency graphs with temporal graphs con-

structed from sequence similarity and uses gated convolutions to capture longer-range temporal dependencies [24]. These methods improve spatial dependency learning but primarily focus on learning unified spatiotemporal representations.

Temporal pattern disentanglement addresses differences among traffic dynamics by organizing encoding around frequency components or periodic structures. Frequency decomposition separates low- and high-frequency components into distinct processing paths for dynamics at different scales. STWave uses a discrete wavelet transform to separate these components and models their spatiotemporal dependencies through two encoding branches [10]. TEDDCF separates trend and event components and assigns each a suitable temporal and spatial encoding path [11]. Periodic structure disentanglement distinguishes regular variation from aperiodic fluctuations [25]. HyperD separates traffic flow into periodic and residual components. It represents periodic patterns through daily and weekly embeddings and spatiotemporal attention, models residual fluctuations in the frequency domain, and constrains the two representations through dual-view alignment [26]. These approaches use different decomposition criteria to match encoding paths to the corresponding dynamics.

Spatiotemporal representation learning and temporal pattern disentanglement primarily seek more expressive base representations. How supplementary information from different sources can further correct predictions when traffic deviates from routine patterns remains less explored.

## 2.2 Heterogeneous Evidence and Expert Modeling

Traffic flow varies across regions and time periods. ST-SSL uses spatial and temporal self-supervised auxiliary tasks to strengthen its representations of this heterogeneity [27]. For structured temporal modeling, ST-ResNet uses three residual branches to capture temporal closeness, periodicity, and trends in regional traffic flow, then learns to fuse their outputs [6].

Other methods explicitly distinguish traffic dynamics through disentanglement or expert mechanisms. D2STGNN separates diffusion and inherent signals through gating and residual decomposition and models the two types of dynamics separately [14]. TESTAM uses temporal, static-graph, and dynamic-graph experts, with gated Top-1 routing to adapt to changing spatiotemporal dependencies [15].

These methods organize heterogeneous dynamics through temporal branches, signal decomposition, or expert routing. We use distinct expert roles to correct a shared base prediction, focusing on the correction role of each evidence source under the current state and the allocation of its contribution.

When traffic deviates from routine patterns, the key question extends beyond which expert to use to the correction roles assigned to different sources of prediction evidence. Directed time-lagged associations describe cross-region historical relations, whereas node self-evolution preserves local continuity. Their different temporal meanings and spatial relations call for distinct processing paths.

## 2.3 Dynamic Dependencies and Historical Message Selection

Traffic forecasting also requires models to account for changing spatial relations and make effective use of cross-region historical information. Attention provides a data-driven approach to learning dynamic dependencies. GMAN uses spatial and temporal attention to model dynamic spatial correlations and temporal dependencies, respectively, and transform attention to connect historical representations with future forecast steps [20]. PDFormer combines spatial self-attention with short- and long-range graph masks to represent dynamic spatial dependencies and uses delay-aware feature transformations to address temporal misalignment in spatial information propagation [16]. TrGNN extracts transition patterns from vehicle trajectories to support road traffic flow forecasting and examines nonrecurrent flow under peak-hour and exceptional-event conditions [28]. These studies show that traffic relations depend on more than spatial distance; they are also affected by temporal shifts in periodic patterns and propagation delays [16], [29].

Adaptive graph learning infers latent spatial relations from data. MTGNN and AGCRN construct adjacency relations from learnable node embeddings, reducing reliance on predefined graphs [21], [22]. Learned node relations provide a basis for spatial interaction. Using cross-region historical messages additionally requires assessing whether a particular message has predictive value in the current traffic context and at a given forecast horizon.

Memory mechanisms also adapt spatial relations or provide historical patterns. MegaCRN generates graph structures from learnable meta-node memory to represent spatiotemporal heterogeneity [30]. RAST constructs context queries through disentangled spatial and temporal encoding, retrieves fine-grained historical patterns from an external memory bank, and fuses the retrieved information through cross-attention before prediction [31]. By retrieving pattern representations from memory, this framework extends the accessible information to stored historical patterns. Historical information use therefore involves two decisions: where to obtain candidates and how to select and fuse them for the current forecasting task.

The usefulness of cross-region historical information depends on the conditions under which it is used [32]. The same neighboring region can provide different predictive value at different historical lags, temporal scales, and forecast steps. Cross-region modeling must therefore identify both which regions are related and which historical patterns to select, with the degree of selection adapting to the forecasting target.

# 3 Problem Definition

We partition the study area into $N$ regular grid cells and treat each cell as a forecasting node. The regional traffic flow matrix at time $t$ is

$$X_t = \left[x_t^{\text{in}},\, x_t^{\text{out}}\right] \in \mathbb{R}^{N\times 2}, \tag{1}$$

where $x_t^{\text{in}}, x_t^{\text{out}} \in \mathbb{R}^N$ denote the inflow and outflow at each node during the sampling interval. These quantities count trips ending and starting in the corresponding region, respectively [33]. We jointly forecast both flows with a single model, with channels ordered as inflow

followed by outflow. Throughout this paper, sequence tensors follow the order of time, node, and flow channel, with the batch dimension omitted.

Given a historical observation sequence ending at time $t$ with length $T$,

$$X_{t-T+1:t} = \left[X_{t-T+1}, \ldots, X_t\right] \in \mathbb{R}^{T\times N\times 2}, \tag{2}$$

the goal is to jointly forecast inflow and outflow over the next $H$ time steps:

$$Y_{t+1:t+H} = \left[X_{t+1}, \ldots, X_{t+H}\right] \in \mathbb{R}^{H\times N\times 2}. \tag{3}$$

Here, $T$ and $H$ denote the historical window length and the number of forecast steps, respectively. We use a 5-minute sampling interval and set $T = H = 12$, using the past 60 minutes to forecast traffic flow over the next 60 minutes.

We formulate regional traffic flow forecasting as learning the spatiotemporal mapping

$$\widehat{Y}_{t+1:t+H} = f_\theta(X_{t-T+1:t}, C_t; \mathcal{S}, \mathcal{B}) \in \mathbb{R}^{H\times N\times 2}, \tag{4}$$

where $\theta$ denotes the learnable model parameters. The temporal information $C_t \in \mathbb{R}^{T\times 2}$ contains the time of day and day of week for the historical window and is shared across nodes. The preconstructed spatial supports $\mathcal{S}$ include self-relations, directed time-lagged associations constructed separately for each flow channel, and candidate cross-region neighborhoods. The lookup table $\mathcal{B}$ stores the routine-flow reference, which is constructed from training data and then fixed. Forecasts use observations up to $t$ and their corresponding temporal information; $\widehat{Y}_{t+1:t+H}$ denotes predictions on the original traffic flow scale.

To describe the observed traffic state's deviation from routine patterns, we consider each time $u \in \{t-T+1, \ldots, t\}$ in the historical window. We retrieve the corresponding routine-flow reference from $\mathcal{B}$:

$$b_u = \left[b_u^{\text{in}},\, b_u^{\text{out}}\right] \in \mathbb{R}^{N\times 2}. \tag{5}$$

We estimate this reference from training observations grouped by node, flow channel, weekday or weekend, and time-of-day slot. The flow deviation from this routine-flow reference is

$$D_u = X_u - b_u \in \mathbb{R}^{N\times 2}. \tag{6}$$

The deviation $D_u$ quantifies the difference between observed flow and its corresponding routine level. Section 4 explains how historical trajectories and the routine-flow reference jointly provide evidence of trend deviation and short-term fluctuation.

Within this joint inflow and outflow forecasting task, we focus on correcting predictions when traffic deviates from routine patterns. We combine recent node dynamics with cross-region time-lagged associations to adaptively allocate correction contributions from heterogeneous evidence. We also select historical messages according to source and target traffic backgrounds and the forecast horizon. Section 4 details the corresponding residual refinement mechanism.

# 4 Methodology

## 4.1 Overall Framework

Figure 2 shows the overall architecture of RBSRN. The framework first generates a base prediction through shared dual-frequency spatiotemporal encoding. The State-Evidence Expert Residual Refinement Module (SERR) organizes correction evidence through self-evolution and directed time-lagged experts. Background-conditioned Cross-region Message Selection (BCMS) selects cross-region historical messages according to source and target traffic backgrounds and the forecast horizon. The two modules read the shared representations in parallel, generate complementary corrections, and combine them additively at the prediction stage:

$$\widetilde{Y} = \widetilde{Y}^0 + \Delta\widetilde{Y}^S + \Delta\widetilde{Y}^B. \tag{7}$$

Here, $\widetilde{Y}^0$ is the base prediction, and $\Delta\widetilde{Y}^S$ and $\Delta\widetilde{Y}^B$ are the SERR and BCMS corrections, respectively. They correspond to $Y_{\text{Base}}$, $\Delta$State, and $\Delta$Message in the figure. All three belong to $\mathbb{R}^{H\times N\times 2}$ and use the scale standardized separately for each flow channel, as indicated by the tilde. Below, we reindex historical positions as $k = 1, \ldots, T$, denote forecast steps by $h = 1, \ldots, H$, and distinguish flow channels using $c \in \{\text{in}, \text{out}\}$.

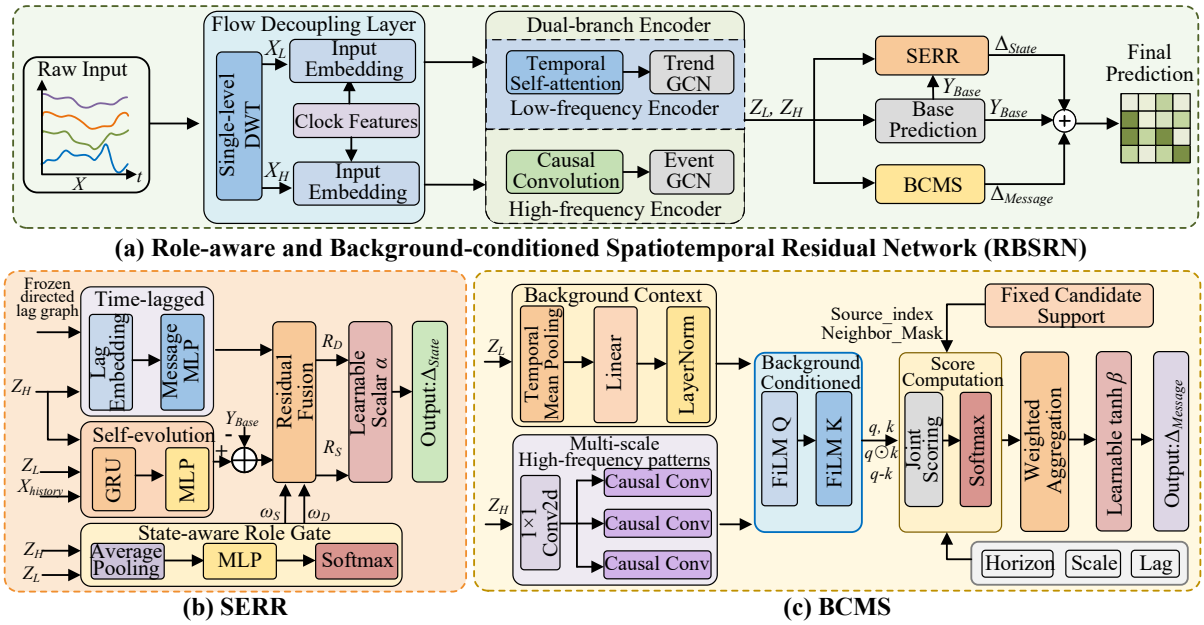



**Figure 2** Overall architecture of RBSRN. (a) Base prediction and parallel residual fusion. (b) SERR expert residual refinement. (c) BCMS background-conditioned historical message selection.

## 4.2 Shared Dual-Frequency Spatiotemporal Representations

To distinguish smooth changes from short-term fluctuations, RBSRN extracts shared representations through low- and high-frequency branches. We first apply single-level discrete wavelet decomposition and reconstruction along the historical time dimension of each flow channel, producing low- and high-frequency sequences. We standardize both sequences using the channel-wise means and standard deviations of training traffic flow. We then append time-of-day and day-of-week information to obtain $U_L, U_H \in \mathbb{R}^{T\times N\times 4}$. The two branches share

the input projection and temporal embeddings. The low-frequency branch applies temporal self-attention and trend graph encoding, whereas the high-frequency branch applies temporal convolution and event graph encoding:

$$Z_L = \mathrm{F}_L(U_L), \tag{8}$$

$$Z_H = \mathrm{F}_H(U_H), \tag{9}$$

where $Z_L, Z_H \in \mathbb{R}^{T\times N\times d_b}$ and $d_b$ denotes the shared representation dimension. We fuse the two feature streams using a gated linear unit (GLU) and a rectified linear unit (ReLU) to obtain the base prediction:

$$\widetilde{Y}^0 = \mathrm{R}(\mathrm{ReLU}[Z_L + \mathrm{GLU}(Z_L) + Z_H]) \,. \tag{10}$$

The convolutional prediction head R spans the full historical window and maps the features to inflow and outflow predictions for the next $H$ steps. SERR and BCMS use $Z_L$ before the GLU operation, together with $Z_H$, as their shared inputs.

## 4.3 State-Evidence Expert Residual Refinement (SERR)

SERR organizes two complementary experts around node self-evolution and cross-region time-lagged associations, then allocates their correction contributions according to the current state (Figure 2(b)). To connect the routine-flow reference with recent dynamics, let $x_{k,i,c}$ denote the raw flow at historical position $k$, node $i$, and channel $c$. The corresponding routine-flow reference estimated from training data is $b_{k,i,c}$. We define the state evidence as

$$\begin{aligned} \bar{x}_{k,i,c} &= \frac{1}{\ell_k} \sum_{r=k-\ell_k+1}^{k} x_{r,i,c}, \qquad \ell_k = \min(3,k), \\ e_{i,c} &= \left[\bar{x}_{T,i,c} - b_{T,i,c},\ x_{T,i,c} - \bar{x}_{T,i,c}\right]^{\mathsf{T}} . \end{aligned} \tag{11}$$

The two components of $e_{i,c} \in \mathbb{R}^2$ describe trend deviation and short-term fluctuation at the final historical time step, respectively. The mean uses only observed positions within the current historical window and adjusts its length to the available observations at the window's start.

The self-evolution expert encodes recent trajectories within each individual node. For each flow channel, it concatenates the sum of the shared dual-frequency features with raw flow and feeds the result into a shared gated recurrent unit (GRU). It then maps the final hidden state to a multi-step prediction:

$$P^{\text{self}}_{:,i,c} = \mathcal{P}_S\big(\psi_S\big(\mathrm{GRU}_{\text{last}}\left([Z_{L,k,i} + Z_{H,k,i} \,\|\, x_{k,i,c}]_{k=1}^{T}\right)\big)\big) \,. \tag{12}$$

Here, $\|$ denotes feature concatenation, and $\psi_S$ is a two-layer perceptron with tanh activation. The projection $\mathcal{P}_S$ linearly maps $d_b$-dimensional features to predictions over $H$ steps. This branch further extracts each node's temporal evolution from the shared spatiotemporal representations.

The directed time-lagged expert retrieves historical features from source nodes through the preconstructed and frozen association set $\mathcal{E}_c$. For an edge $(j, i, \ell) \in \mathcal{E}_c$, $j$ is the source node, $i$ is the target node, and $\ell$ is a historical lag satisfying $0 < \ell < T$. A lag embedding modulates the source high-frequency features, which are then combined with the target state evidence to form

supplementary messages:

$$\begin{aligned} \eta_{i,c} &= W_e e_{i,c}, \\ m_{j\to i,\ell,c} &= \psi_D([(1+\tanh v_\ell)\odot Z_{H,T-\ell,j} \,\|\, \eta_{i,c}]), \\ R^{\text{delay}}_{:,i,c} &= \mathcal{P}_D\left(\eta_{i,c} + \sum_{(j,i,\ell)\in\mathcal{E}_c} a^c_{ji\ell} m_{j\to i,\ell,c}\right). \end{aligned} \tag{13}$$

Here, $W_e \in \mathbb{R}^{d_b\times 2}$ projects the evidence, and $v_\ell \in \mathbb{R}^{d_b}$ is a learnable lag embedding. The message transformation $\psi_D$ uses a sigmoid linear unit (SiLU) activation without bias, $a^c_{ji\ell}$ is a fixed edge weight, and $\mathcal{P}_D$ is a multi-step linear output head. Each expert shares its own parameters across nodes and flow channels, while directed time-lagged associations are specified separately for each flow channel.

A state-conditioned gate determines expert contributions from the temporal means of low- and high-frequency features and the state evidence for both flow channels:

$$\Omega_i = \text{Softmax}_{\text{expert}}\big(\text{reshape}_{2\times 2}\ g\big([\bar{Z}_{L,i} \,\|\, \bar{Z}_{H,i} \,\|\, e_{i,\text{in}} \,\|\, e_{i,\text{out}}]\big)\big). \tag{14}$$

The temporal means $\bar{Z}_{L,i}, \bar{Z}_{H,i}$ are $d_b$-dimensional vectors averaged over the historical window, and $g$ is a two-layer perceptron with SiLU activation. In $\Omega_i \in \mathbb{R}^{2\times 2}$, rows correspond to flow channels, and columns correspond to the self-evolution and directed time-lagged experts. Softmax normalizes along the expert dimension to produce $\omega^S_{i,c}$ and $\omega^D_{i,c}$. These weights vary by sample, node, and flow channel but are shared across all forecast steps. The resulting residual correction is

$$\Delta\widetilde{Y}^S_{h,i,c} = \alpha\left[\omega^S_{i,c}\left(P^{\text{self}}_{h,i,c} - \widetilde{Y}^0_{h,i,c}\right) + \omega^D_{i,c} R^{\text{delay}}_{h,i,c}\right]. \tag{15}$$

Both $P^{\text{self}}$ and $R^{\text{delay}}$ use the same standardized flow scale as the base prediction, and $\alpha$ is a global learnable scalar. The self-evolution expert contributes a residual relative to the base prediction, whereas the directed time-lagged expert provides a correction from cross-region historical information. The state-conditioned gate combines the contributions of these two experts.

## 4.4 Background-conditioned Cross-region Message Selection (BCMS)

BCMS selects historical messages within fixed candidate neighborhoods according to the current background and forecast horizon (Figure 2(c)). It first maps the temporal mean of low-frequency features to a node background representation and extracts multiscale patterns from high-frequency features:

$$\xi_i = \text{LN}\big(\mathcal{P}_b(\bar{Z}_{L,i})\big), \qquad z^s = \mathcal{T}_s(\mathcal{P}_H(Z_H)). \tag{16}$$

Here, $\xi_i \in \mathbb{R}^d$ and $z^s \in \mathbb{R}^{T\times N\times d}$, with $d = 32$. The operator $\mathcal{P}_b$ is a linear projection, LN denotes layer normalization, and $\mathcal{P}_H$ is a $1\times 1$ convolution shared across scales. Each $\mathcal{T}_s$ consists of a left-padded depthwise temporal convolution, a pointwise convolution, and a Gaussian error linear unit (GELU) activation. The three scales use a kernel size of 3 and dilation rates of 1, 2, and 4, yielding temporal spans $r_s \in \{3, 5, 9\}$.

For node $i$, let $\mathcal{N}_i$ denote its fixed candidate source set and consider historical lags $\tau \in \{0, 1, 2, 3\}$. The valid candidate set is $\mathcal{A}_i = \{(j, \tau, s) : j \in \mathcal{N}_i,\ r_s + \tau \leq T\}$, with at most 8 sources retained per node. At each scale, the query represents the target's final historical pattern, and the key represents a source historical pattern. Their respective low-frequency backgrounds modulate the query and key:

$$\begin{aligned} q_i^s &= (1 + \gamma_q(\xi_i)) \odot Q(z_{T,i}^s) + \delta_q(\xi_i), \\ k_{j,\tau}^s &= (1 + \gamma_k(\xi_j)) \odot K(z_{T-\tau,j}^s) + \delta_k(\xi_j). \end{aligned} \tag{17}$$

The operators $Q, K$ are learnable affine mappings; $\gamma$ and $\delta$ produce $d$-dimensional scaling and shifting parameters, respectively [34]. Thus, both source and target low-frequency traffic backgrounds condition the matching of high-frequency patterns.

For forecast step $h$, we define the joint score as

$$\begin{aligned} u_{i,j,\tau,s} &= A_q q_i^s + A_k k_{j,\tau}^s + A_p(q_i^s \odot k_{j,\tau}^s) + A_d(q_i^s - k_{j,\tau}^s) + b_a, \\ a_{h,i,j,\tau,s} &= w_a^\top \,\mathrm{GELU}(u_{i,j,\tau,s} + e_{\text{lag}}(\tau) + e_{\text{scale}}(r_s) + e_{\text{hor}}(h))\,. \end{aligned} \tag{18}$$

Here, $A_q, A_k, A_p, A_d \in \mathbb{R}^{d\times d}$ and $b_a, w_a \in \mathbb{R}^d$ are learnable parameters. Each of the three temporal encodings applies a bias-free linear projection to $[v, \sin(2\pi v), \cos(2\pi v)]^\top$. The value $v$ expresses the corresponding historical lag, temporal span, or forecast horizon in hours. Introducing the forecast-step encoding before nonlinear scoring allows candidate contributions to vary with the forecast horizon.

To control the overall contribution of historical information, we add a zero-message option with both score and value set to zero at each forecast step. For each actual candidate, joint normalization gives the weight

$$w_{h,i,j,\tau,s} = \frac{\exp(a_{h,i,j,\tau,s})}{1 + \displaystyle\sum_{(j',\tau',s')\in\mathcal{A}_i} \exp(a_{h,i,j',\tau',s'})}. \tag{19}$$

The denominator's constant 1 corresponds to the zero-message option. Its probability mass allows the model to reduce the total weight assigned to actual historical messages. We project the source high-frequency patterns to values, aggregate them with the selection weights, and generate corrections for both flow channels:

$$\begin{aligned} m_{h,i} &= \sum_{(j,\tau,s)\in\mathcal{A}_i} w_{h,i,j,\tau,s} W_v z_{T-\tau,j}^s, \\ \Delta\widetilde{Y}_{h,i,:}^B &= \tanh(\beta)\, W_o m_{h,i}. \end{aligned} \tag{20}$$

Both $W_v \in \mathbb{R}^{d\times d}$ and $W_o \in \mathbb{R}^{2\times d}$ are bias-free projections, and $\beta$ is a global learnable scalar. Inflow and outflow share the message selection weights and aggregated representation, while the two-output head produces their respective corrections. BCMS generates weights for each forecast step, complementing the SERR expert weights shared across forecast steps.

## 4.5 Joint Forecasting Output

We add both corrections to the base prediction according to Eq. (7). We then restore the original traffic flow scale using fixed channel-wise statistics estimated from training input win-

dows:

$$\widehat{Y}_{t+h,i,c} = \sigma_c \left( \widetilde{Y}^0_{h,i,c} + \Delta\widetilde{Y}^S_{h,i,c} + \Delta\widetilde{Y}^B_{h,i,c} \right) + \mu_c. \tag{21}$$

Here, $\mu_c$ and $\sigma_c$ denote the mean and standard deviation of channel $c$, respectively. We initialize both global correction-strength parameters, $\alpha$ and $\beta$, to zero, so the initial output equals the base prediction. We also zero-initialize the final SERR gating layer, giving both experts an initial weight of 0.5. The model then learns the contributions of the two complementary correction pathways under a joint inflow and outflow forecasting objective.

# 5 Experiments and Analysis

## 5.1 Experimental Settings

### 5.1.1 Datasets, Preprocessing, and Forecasting Protocol

We conducted regional traffic flow forecasting experiments on two datasets from Chengdu and Shanghai. These datasets contain Chengdu DiDi ride-hailing orders collected in November 2016 and Shanghai taxi origin–destination (OD) records collected in August 2016, respectively. We selected a fixed core area in each city and divided it into a regular $32 \times 32$ grid. Inflow and outflow are measured as counts per 5 min. Each destination contributes to the inflow of its grid cell at arrival time, and each origin contributes to outflow at departure time. The two endpoint types were aggregated independently. Table 1 summarizes dataset properties and sample counts. Figures 3 and 4 show the endpoint distributions, grid partitions, and mean flow distributions for the two cities.

Both cities followed the same order-cleaning and flow-aggregation procedure, with all timestamps expressed in Beijing time (UTC+8). We first removed exact duplicates and records with invalid timestamps or coordinates. Retained orders must last 2–120 minutes, have both endpoints within the city's coarse geographic boundary, and have an average speed no greater than $120\,\mathrm{km/h}$. Average speed is calculated from the great-circle distance between endpoints and the trip duration. Endpoints within the core area and observation period were then aggregated into their grid cells and time intervals. Locations with valid source quality and no endpoint events were assigned zero. In Shanghai, source quality was flagged as invalid for 2016-08-11 to 2016-08-14. Records from this period were retained in the forecasting data and in the statistics used to standardize training inputs.

For each dataset, we generated sliding windows with a stride of 1 step (5 minutes). Each window uses the past 12 observations to jointly predict inflow and outflow over the next 12 steps, covering 60 minutes of history and 60 minutes of forecasts. We retained the time-of-day and day-of-week information associated with each historical window. Only windows with complete historical inputs and forecast targets were included. We split these windows chronologically by forecast origin into training, validation, and test sets at an approximate ratio of 6:2:2. Table 1 lists the window counts for each split. Splitting occurred after window generation, and no windows were removed at split boundaries. Both the training–validation and validation–test boundaries

**Table 1** Dataset properties and sample splits

| Property | Chengdu | Shanghai |
|---|---|---|
| Observation period | 2016-11-01 to 2016-11-30 | 2016-08-01 to 2016-08-31 |
| Grid size | $32 \times 32$ | $32 \times 32$ |
| Approx. cell dimensions (km) | $0.42 \times 0.44$ | $0.80 \times 0.68$ |
| Sampling interval | 5 minutes | 5 minutes |
| Complete time intervals | 8,640 | 8,928 |
| Orders after cleaning | 6,082,753 | 7,744,423 |
| Training windows | 5,170 | 5,343 |
| Validation windows | 1,724 | 1,781 |
| Test windows | 1,723 | 1,781 |

therefore shared 11 historical time steps and 11 forecast-target time steps. All methods used the same split indices and forecasting task.

Standardization means and standard deviations were computed separately for each flow channel from the historical inputs of training windows and fixed for validation and testing. Predictions were restored to the original flow scale by channel-wise inverse standardization. Section 5.1.3 describes model input processing and validation-based checkpoint selection. Section 5.1.2 defines the evaluation metrics and their aggregation.

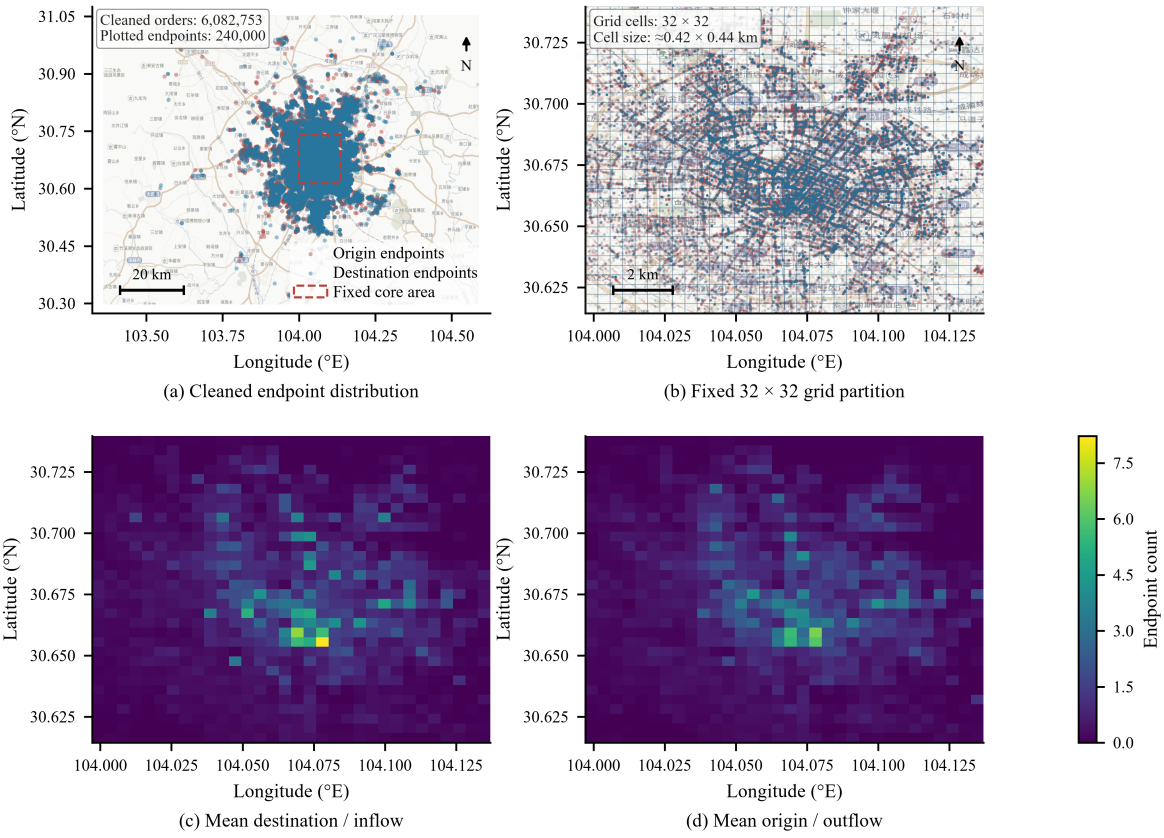



**Figure 3** Endpoint distribution, grid partition, and mean flow distribution of the Chengdu dataset. Mean flows were calculated only over 5-minute intervals with valid source quality.

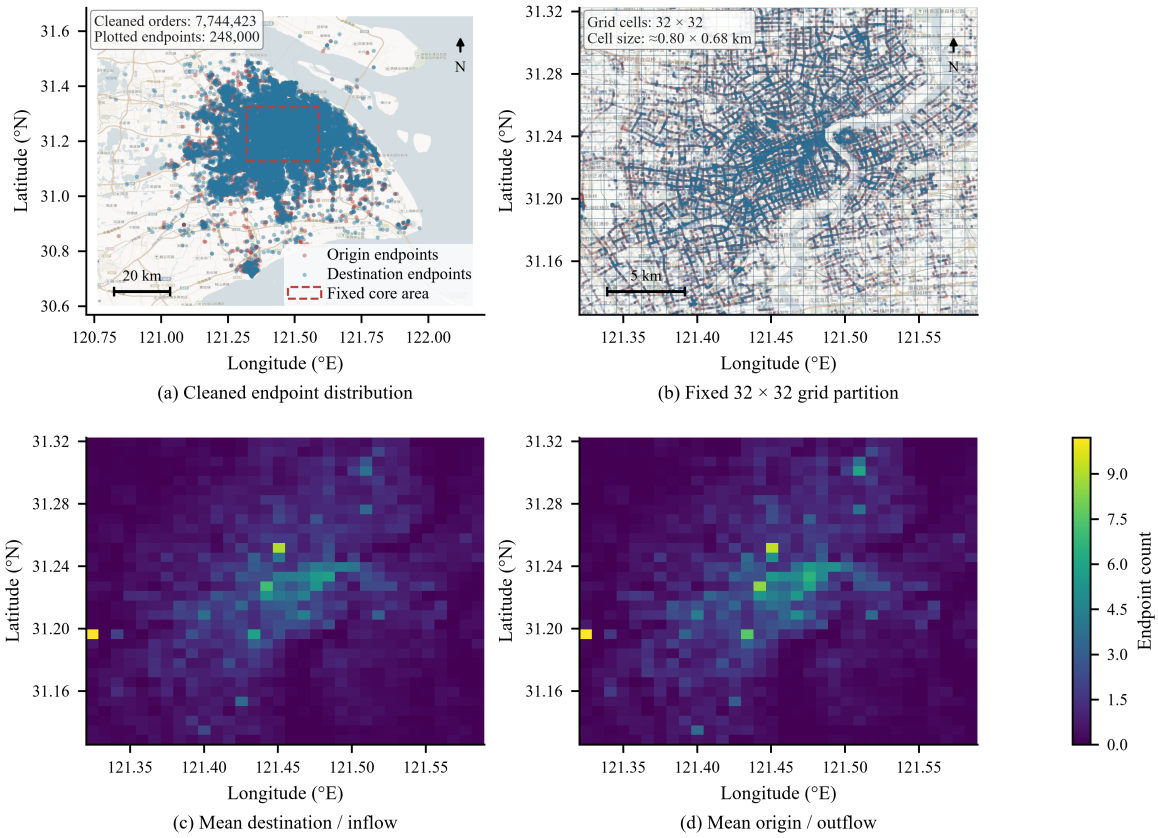



**Figure 4** Endpoint distribution, grid partition, and mean flow distribution of the Shanghai dataset. Mean flows were calculated only over 5-minute intervals with valid source quality.

### 5.1.2 Baseline Methods and Evaluation Metrics

The main experiments compared RBSRN with 13 baseline methods spanning three forecasting paradigms. Conventional methods include historical average (HA) and support vector regression (SVR). Deep learning models for traffic forecasting include DCRNN [7], STGCN [8], Graph WaveNet (GWNet) [9], DSTAGNN [35], PDFormer [16], STDN [36], MetaDG [37], HyperD [26], and RAST [31]. These models cover graph structure learning, dynamic spatiotemporal dependencies, sequence decomposition, and retrieval of historical information. General time-series models include PatchTST [38] and TimeMixer [39]. All methods followed the fixed chronological split and 12-step forecasting task in Section 5.1.1. Inflow and outflow were evaluated separately on Chengdu and Shanghai.

The scenario analysis of deviations from routine patterns additionally included vector autoregression (VAR), STNorm [40], and STWave [10], bringing the comparison to 16 baseline methods.

We use mean absolute error (MAE) and root mean squared error (RMSE), both computed on the original flow scale after inverse standardization. The main results tables use a positive-target mask, evaluating locations where the observed flow is greater than zero. The evaluation-scope sensitivity analysis used an all-grid evaluation of test predictions from the same model checkpoint, including locations with zero targets. Let $\mathcal{T}_{\text{test}}$ denote the set of forecast origins for the actual test windows. Let $y_{t,h,i,c}$ and $\widehat{y}_{t,h,i,c}$ denote the observed and predicted values for

forecast origin $t$, forecast step $h$, grid cell $i$, and flow channel $c \in \{\text{in}, \text{out}\}$, respectively. The prediction error is $e_{t,h,i,c} = \widehat{y}_{t,h,i,c} - y_{t,h,i,c}$. The two evaluation scopes are defined as

$$\Omega^{+}_{h,c} = \big\{(t,i) \mid t \in \mathcal{T}_{\text{test}},\ 1 \le i \le N,\ y_{t,h,i,c} > 0\big\},$$
$$\Omega^{\text{all}}_{h,c} = \mathcal{T}_{\text{test}} \times \{1, \dots, N\}.$$

For $m \in \{+, \text{all}\}$, we pool all test windows and nodes within the corresponding scope to compute

$$\text{MAE}^{m}_{h,c} = \frac{1}{|\Omega^{m}_{h,c}|} \sum_{(t,i)\in\Omega^{m}_{h,c}} |e_{t,h,i,c}|, \tag{22}$$

$$\text{RMSE}^{m}_{h,c} = \sqrt{\frac{1}{|\Omega^{m}_{h,c}|} \sum_{(t,i)\in\Omega^{m}_{h,c}} e^{2}_{t,h,i,c}}. \tag{23}$$

Here, $|\Omega^{m}_{h,c}|$ is the number of evaluated locations. Both scopes exclude duplicate windows introduced by test-batch padding, and inflow and outflow are evaluated separately.

We report the single-step results H3, H6, and H12, corresponding to forecast steps 3, 6, and 12, or forecast horizons of 15, 30, and 60 minutes. AVG12 denotes the arithmetic mean of the metrics over all 12 forecast steps:

$$\text{AVG12-}\mathcal{M}^{m}_{c} = \frac{1}{12} \sum_{h=1}^{12} \mathcal{M}^{m}_{h,c}, \qquad \mathcal{M} \in \{\text{MAE}, \text{RMSE}\}. \tag{24}$$

For AVG12 RMSE, we first compute RMSE at each forecast step and then average these values across steps.

### 5.1.3 Implementation and Computing Environment

Model experiments used a single NVIDIA A100-SXM4-40GB GPU. Table 2 summarizes the hardware, software, and training settings for the hyperparameter experiments. All configurations in the hyperparameter sweep used the same data split, with a maximum of 200 training epochs and an early-stopping patience of 20 epochs. We selected the checkpoint with the lowest joint validation MAE and reported both MAE and RMSE at that checkpoint.

**Table 2** Model computing environment and training settings

| Item | Setting |
|---|---|
| Optimizer and learning rate | Ranger; fixed at 0.001, with no learning-rate schedule |
| Weight decay | $10^{-4}$; no decay for the BCMS parameter $\beta$ |
| Batch size | 64 for both training and validation |
| Training budget and checkpoint selection | At most 200 epochs, with an early-stopping patience of 20 epochs; checkpoint selected by the lowest joint validation MAE |
| GPU | One NVIDIA A100-SXM4-40GB, with 40 GB of GPU memory |
| CPU | Two Intel Xeon Gold 6138 processors @ 2.00 GHz, with 40 cores and 80 threads in total |
| Software environment | Python 3.11.15; PyTorch 2.7.1+cu128; CUDA 12.8 (PyTorch build version) |

## 5.2 Main Experimental Results

This section evaluates RBSRN in terms of overall prediction accuracy, spatial distributions, deviations from routine patterns, and the effects of residual corrections. Comparisons across the full test set and fixed-model diagnostics examine how the two types of correction evidence perform under different traffic conditions.

### 5.2.1 Overall Prediction Performance

Tables 3–6 compare RBSRN with 13 baseline methods on inflow and outflow forecasting in Chengdu and Shanghai, using positive-target evaluation. Relative changes were calculated from unrounded values against the strongest baseline method for each task and metric. Bold and underlined values indicate the best and second-best results in each column, respectively.

RBSRN achieved the lowest AVG12 MAE and RMSE on all four tasks. For Chengdu inflow, Chengdu outflow, Shanghai inflow, and Shanghai outflow, AVG12 MAE decreased by 2.00%, 0.46%, 1.18%, and 0.43% relative to each task's strongest baseline method. AVG12 RMSE decreased by 5.01%, 1.50%, 2.33%, and 0.06%, respectively, with STGCN, STGCN, HyperD, and HyperD as the corresponding strongest baseline methods.

Across forecast horizons, RBSRN achieved the lowest MAE and RMSE at H3, H6, and H12 for inflow forecasting in both cities. For outflow forecasting, HyperD achieved the best H12 MAE in both cities and the best H12 RMSE in Shanghai. RBSRN achieved the lowest errors in all remaining reported comparisons.

The proportion of zero targets is calculated over the same test evaluation positions. For channel $c$, it is defined as

$$\rho_c = 1 - \frac{\sum_{h=1}^{12} |\Omega_{h,c}^{+}|}{12N|\mathcal{T}_{\text{test}}|}. \tag{25}$$

The denominator counts the evaluation positions across all actual test windows, 12 forecast steps, and 1,024 grid cells. When the same timestamp appears in multiple forecast windows, each

**Table 3** Inflow forecasting results for Chengdu ($32 \times 32$ grid)

| | H3 | | H6 | | H12 | | AVG12 | |
|---|---|---|---|---|---|---|---|---|
| Model | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE |
| HA | 1.053 | 1.522 | 1.083 | 1.600 | 1.144 | 1.739 | 1.088 | 1.609 |
| SVR | 0.810 | 1.354 | 0.835 | 1.391 | 0.892 | 1.499 | 0.840 | 1.405 |
| DCRNN (ICLR'18) | 0.725 | 1.321 | 0.790 | 1.363 | 1.025 | 1.574 | 0.829 | 1.399 |
| STGCN (IJCAI'18) | 0.721 | 1.342 | 0.724 | 1.348 | 0.743 | 1.409 | 0.728 | 1.362 |
| GWNet (IJCAI'19) | 0.719 | 1.332 | 0.731 | 1.373 | 0.756 | 1.450 | 0.733 | 1.378 |
| DSTAGNN (ICML'22) | 1.022 | 1.461 | 1.008 | 1.458 | 1.044 | 1.505 | 1.027 | 1.473 |
| PDFormer (AAAI'23) | 1.002 | 1.443 | 1.013 | 1.473 | 1.030 | 1.515 | 1.014 | 1.473 |
| PatchTST (ICLR'23) | 0.799 | 1.405 | 0.826 | 1.483 | 0.880 | 1.614 | 0.831 | 1.490 |
| TimeMixer (ICLR'24) | 0.818 | 1.434 | 0.844 | 1.505 | 0.895 | 1.620 | 0.848 | 1.510 |
| STDN (AAAI'25) | 1.178 | 1.589 | 1.180 | 1.593 | 1.179 | 1.600 | 1.178 | 1.593 |
| MetaDG (AAAI'26) | 1.102 | 1.564 | 1.170 | 1.678 | 1.291 | 1.882 | 1.178 | 1.692 |
| HyperD (AAAI'26) | 0.723 | 1.369 | 0.723 | 1.369 | 0.726 | 1.382 | 0.723 | 1.369 |
| RAST (AAAI'26) | 0.724 | 1.332 | 0.734 | 1.372 | 0.756 | 1.446 | 0.736 | 1.377 |
| **RBSRN (Ours)** | **0.704** | **1.282** | **0.707** | **1.290** | **0.717** | **1.318** | **0.708** | **1.294** |

**Table 4** Outflow forecasting results for Chengdu ($32 \times 32$ grid)

| | H3 | | H6 | | H12 | | AVG12 | |
|---|---|---|---|---|---|---|---|---|
| Model | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE |
| HA | 1.023 | 1.404 | 1.043 | 1.439 | 1.085 | 1.510 | 1.047 | 1.445 |
| SVR | 0.792 | 1.239 | 0.809 | 1.251 | 0.838 | 1.285 | 0.810 | 1.255 |
| DCRNN (ICLR'18) | 0.714 | 1.230 | 0.761 | 1.237 | 1.151 | 1.642 | 0.844 | 1.326 |
| STGCN (IJCAI'18) | 0.714 | 1.235 | 0.716 | 1.239 | 0.725 | 1.258 | 0.718 | 1.242 |
| GWNet (IJCAI'19) | 0.713 | 1.253 | 0.718 | 1.265 | 0.729 | 1.289 | 0.719 | 1.267 |
| DSTAGNN (ICML'22) | 1.018 | 1.383 | 1.007 | 1.376 | 1.035 | 1.410 | 1.021 | 1.390 |
| PDFormer (AAAI'23) | 0.987 | 1.365 | 0.990 | 1.374 | 0.997 | 1.391 | 0.990 | 1.374 |
| PatchTST (ICLR'23) | 0.786 | 1.300 | 0.802 | 1.333 | 0.837 | 1.396 | 0.805 | 1.337 |
| TimeMixer (ICLR'24) | 0.807 | 1.315 | 0.823 | 1.345 | 0.856 | 1.404 | 0.826 | 1.349 |
| STDN (AAAI'25) | 1.135 | 1.501 | 1.135 | 1.501 | 1.135 | 1.503 | 1.134 | 1.500 |
| MetaDG (AAAI'26) | 1.082 | 1.471 | 1.139 | 1.543 | 1.245 | 1.672 | 1.146 | 1.550 |
| HyperD (AAAI'26) | 0.712 | 1.249 | 0.712 | 1.249 | **0.714** | 1.254 | 0.712 | 1.249 |
| RAST (AAAI'26) | 0.718 | 1.242 | 0.724 | 1.257 | 0.735 | 1.282 | 0.724 | 1.258 |
| **RBSRN (Ours)** | **0.706** | **1.220** | **0.708** | **1.222** | 0.715 | **1.234** | **0.709** | **1.223** |

corresponding forecasting task is counted separately. Table 7 reports the zero-target proportions and results under both evaluation scopes for each city and channel.

Table 7 compares AVG12 results under positive-target and all-grid evaluation using the same RBSRN checkpoint as the main tables for each city. Zero targets accounted for 64.17%–71.68% of evaluation positions across the four tasks. Including these positions increased MAE and decreased RMSE in every task, showing that evaluation scope affects the two metrics differently. Baseline rankings in the main results use positive-target evaluation.

**Table 5** Inflow forecasting results for Shanghai ($32 \times 32$ grid)

| | H3 | | H6 | | H12 | | AVG12 | |
|---|---|---|---|---|---|---|---|---|
| Model | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE |
| HA | 1.019 | 1.437 | 1.040 | 1.490 | 1.080 | 1.593 | 1.043 | 1.498 |
| SVR | 0.817 | 1.251 | 0.837 | 1.274 | 0.879 | 1.344 | 0.841 | 1.283 |
| DCRNN (ICLR'18) | 0.734 | 1.243 | 0.748 | 1.262 | 0.808 | 1.328 | 0.758 | 1.272 |
| STGCN (IJCAI'18) | 0.741 | 1.257 | 0.743 | 1.260 | 0.754 | 1.293 | 0.745 | 1.268 |
| GWNet (IJCAI'19) | 0.735 | 1.268 | 0.742 | 1.288 | 0.760 | 1.343 | 0.744 | 1.295 |
| DSTAGNN (ICML'22) | 1.010 | 1.410 | 0.950 | 1.360 | 1.009 | 1.426 | 0.994 | 1.402 |
| PDFormer (AAAI'23) | 0.970 | 1.356 | 0.977 | 1.369 | 0.995 | 1.413 | 0.979 | 1.376 |
| PatchTST (ICLR'23) | 0.800 | 1.333 | 0.816 | 1.382 | 0.849 | 1.480 | 0.819 | 1.392 |
| TimeMixer (ICLR'24) | 0.821 | 1.331 | 0.836 | 1.372 | 0.868 | 1.463 | 0.839 | 1.381 |
| STDN (AAAI'25) | 1.219 | 1.651 | 1.220 | 1.659 | 1.219 | 1.683 | 1.218 | 1.661 |
| MetaDG (AAAI'26) | 1.082 | 1.517 | 1.168 | 1.624 | 1.421 | 1.944 | 1.201 | 1.667 |
| HyperD (AAAI'26) | 0.737 | 1.261 | 0.737 | 1.261 | 0.737 | 1.263 | 0.737 | 1.260 |
| RAST (AAAI'26) | 0.736 | 1.245 | 0.740 | 1.259 | 0.749 | 1.289 | 0.741 | 1.261 |
| **RBSRN (Ours)** | **0.726** | **1.225** | **0.727** | **1.229** | **0.733** | **1.243** | **0.728** | **1.231** |

**Table 6** Outflow forecasting results for Shanghai ($32 \times 32$ grid)

| | H3 | | H6 | | H12 | | AVG12 | |
|---|---|---|---|---|---|---|---|---|
| Model | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE |
| HA | 1.059 | 1.528 | 1.079 | 1.559 | 1.118 | 1.629 | 1.082 | 1.565 |
| SVR | 0.866 | 1.373 | 0.882 | 1.395 | 0.914 | 1.447 | 0.883 | 1.398 |
| DCRNN (ICLR'18) | 0.782 | 1.355 | 0.794 | 1.365 | 0.856 | 1.388 | 0.804 | 1.365 |
| STGCN (IJCAI'18) | 0.784 | 1.353 | 0.787 | 1.359 | 0.797 | 1.385 | 0.789 | 1.364 |
| GWNet (IJCAI'19) | 0.783 | 1.381 | 0.788 | 1.394 | 0.801 | 1.424 | 0.789 | 1.396 |
| DSTAGNN (ICML'22) | 1.017 | 1.462 | 0.994 | 1.446 | 1.080 | 1.535 | 1.016 | 1.469 |
| PDFormer (AAAI'23) | 1.010 | 1.459 | 1.017 | 1.472 | 1.034 | 1.511 | 1.019 | 1.478 |
| PatchTST (ICLR'23) | 0.848 | 1.439 | 0.863 | 1.465 | 0.897 | 1.533 | 0.866 | 1.473 |
| TimeMixer (ICLR'24) | 0.869 | 1.448 | 0.883 | 1.473 | 0.915 | 1.538 | 0.886 | 1.480 |
| STDN (AAAI'25) | 1.109 | 1.591 | 1.107 | 1.593 | 1.105 | 1.605 | 1.106 | 1.595 |
| MetaDG (AAAI'26) | 1.127 | 1.600 | 1.245 | 1.748 | 1.599 | 2.238 | 1.298 | 1.825 |
| HyperD (AAAI'26) | 0.778 | 1.344 | 0.778 | 1.344 | **0.778** | **1.347** | 0.778 | 1.344 |
| RAST (AAAI'26) | 0.787 | 1.366 | 0.792 | 1.380 | 0.804 | 1.409 | 0.793 | 1.381 |
| **RBSRN (Ours)** | **0.772** | **1.339** | **0.774** | **1.342** | 0.779 | 1.353 | **0.775** | **1.343** |

**Table 7** Zero-target proportions in the test set and sensitivity of RBSRN to evaluation scope (AVG12)

| City | Channel | Zero targets (%) | Positive targets ($y > 0$) | | All grid cells (including zero targets) | |
|---|---|---|---|---|---|---|
| | | | MAE | RMSE | MAE | RMSE |
| Chengdu | Inflow | 71.68 | 0.708421 | 1.293545 | 0.936965 | 1.117916 |
| | Outflow | 70.13 | 0.708654 | 1.223468 | 0.936962 | 1.105671 |
| Shanghai | Inflow | 64.17 | 0.728085 | 1.230839 | 0.937587 | 1.139290 |
| | Outflow | 64.94 | 0.774534 | 1.342986 | 0.954306 | 1.184390 |

### 5.2.2 Spatial Prediction Performance

To assess how well RBSRN locates high-flow grid cells, we evaluated predictions 30 minutes after the forecast origin (H6). The analysis covered 1,723 test windows in Chengdu and 1,781 in Shanghai. For each window, we calculated MAE, Spearman rank correlation, and hotspot intersection over union (IoU) over all 1,024 grid cells. Each metric was then averaged over the windows for which it was defined. Inflow and outflow were evaluated separately, and total flow was their cellwise sum. MAE used continuous predictions after inverse standardization, measured in counts per 5 min. Rank correlation assigned average ranks to ties; windows with constant observed flow contributed only to MAE and IoU.

Hotspots were defined separately for predicted and observed maps as positive-flow cells within the top $q$ fraction of the flow ranking. The threshold was the $\lceil qN \rceil$-th largest flow value, and all positive values tied at that threshold were included. IoU was the size of the hotspot intersection divided by the size of the union; it was undefined when the union was empty.

**Table 8** Spatial prediction statistics at H6 across the full test set

| City | Flow | Windows | All-grid MAE | Spearman | Valid correlations | $\text{IoU}_{10\%}$ | $\text{IoU}_{5\%}$ |
|---|---|---|---|---|---|---|---|
| Chengdu | Inflow | 1,723 | 0.9366 | 0.4327 | 1,723 | 0.3420 | 0.3495 |
| | Outflow | 1,723 | 0.9367 | 0.4503 | 1,723 | 0.3356 | 0.3147 |
| | Total | 1,723 | 1.6750 | 0.5324 | 1,723 | 0.4155 | 0.4142 |
| Shanghai | Inflow | 1,781 | 0.9382 | 0.4524 | 1,760 | 0.3224 | 0.3106 |
| | Outflow | 1,781 | 0.9549 | 0.4920 | 1,759 | 0.3547 | 0.3502 |
| | Total | 1,781 | 1.6722 | 0.5543 | 1,767 | 0.4068 | 0.4117 |

Table 8 reports mean spatial rank correlations for total flow of 0.5324 in Chengdu and 0.5543 in Shanghai. At the top 10% threshold, cells identified as hotspots in both maps accounted for 41.55% and 40.68% of their union on average, respectively. For inflow or outflow evaluated separately, this proportion was 32.24%–35.47%. Tightening the threshold to the top 5% left the mean total-flow IoU at approximately 0.41 in both cities. Variation between windows at the same horizon was larger. The interquartile ranges of H6 total-flow $\text{IoU}_{10\%}$ were 0.270–0.537 in Chengdu and 0.331–0.519 in Shanghai. By comparison, the ranges of mean values over H1–H12 were 0.4122–0.4174 and 0.4048–0.4087, respectively.

Figure 5 shows the spatial distribution of total flow at 08:00 on each city's first complete H6 target day. The Shanghai example had a hotspot IoU of 0.4718. Its predicted maximum flow was 23.02 counts per 5 min, exceeding the observed maximum of 18 counts per 5 min. In this window, predicted and observed hotspots partially overlapped, but the model overestimated the observed peak flow.

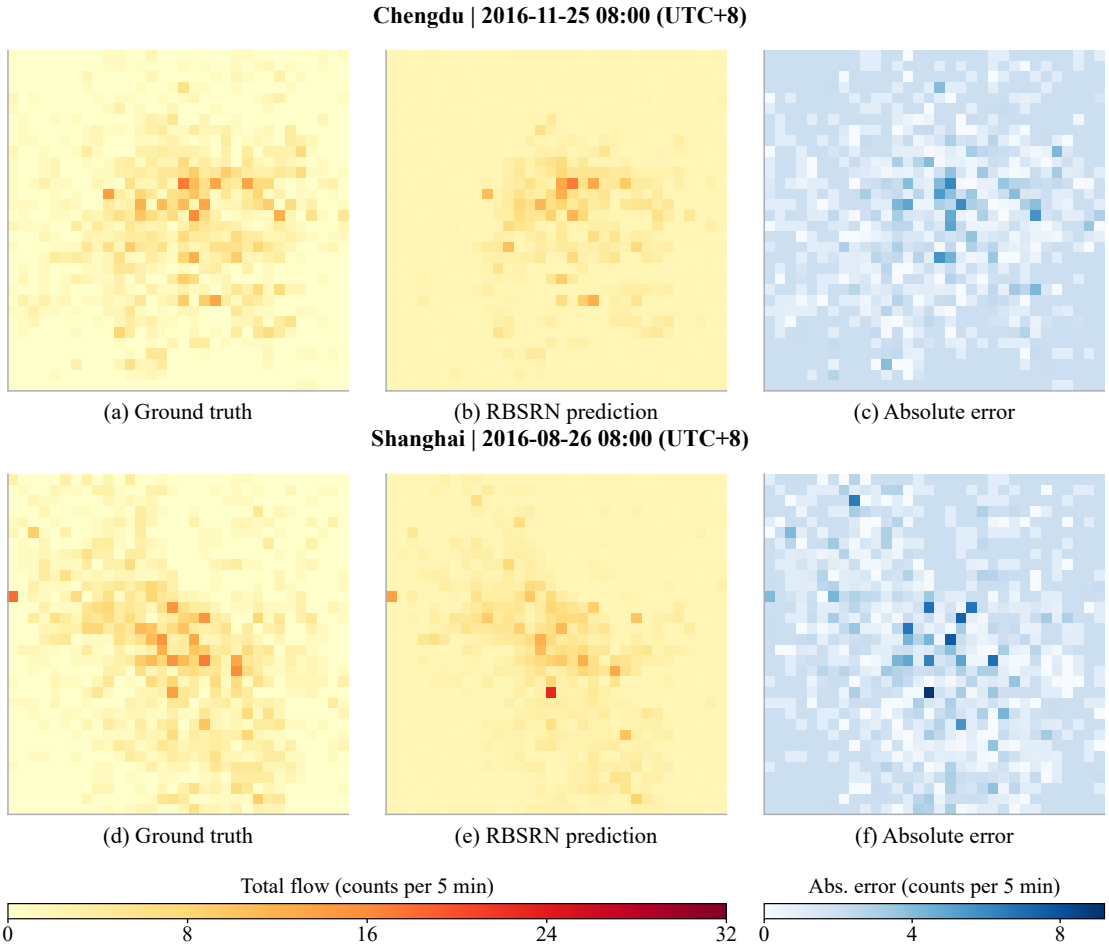



**Figure 5** Example spatial predictions of total flow at H6 (08:00). The top and bottom rows show Chengdu and Shanghai, respectively. The columns show observations, predictions, and absolute errors.

### 5.2.3 Prediction Performance across Levels of Deviation from Routine Patterns

All-grid spatial statistics characterize the distribution of traffic flow at a given time. To examine prediction error when recent flow deviates from routine levels, this section groups observations by window, grid cell, and channel. At positions with positive H6 target flow, RBSRN was compared with 16 baseline methods. The Chengdu scenario analysis used the same checkpoint as the subsequent residual diagnostics. Its full-test-set H6 MAE was 0.709952 for inflow and 0.709944 for outflow; the Chengdu main tables in Section 5.2.1 use a different training run. The Shanghai scenario performance and residual diagnostics both used the epoch-115 checkpoint corresponding to the main tables.

The routine-flow reference is the median training flow for the same grid cell, channel, and 5-minute time slot, separately for weekdays and weekends. Only deduplicated training-history timestamps that pass quality checks are used. If a time slot has fewer than 4 observations for one day type, the two day types are pooled. If that slot has no observations, the training median for the grid cell and channel is used. The difference between each observation and its reference is divided by a robust scale to obtain the standardized residual $r_k$. This scale is the median absolute deviation of training residuals multiplied by 1.4826, with a lower bound of 1 count.

The deviation score $D = \frac{1}{12}\sum_{k=1}^{12}|r_k|$ uses only the input window. For each city and channel, all grid cells are pooled from training windows whose 12 historical steps all pass quality checks. The 50th, 80th, and 95th percentiles of $D$ define four groups (Table 9); observations equal to a boundary enter the lower group.

**Table 9** Scenario-group thresholds derived from training data in the two cities

| City | Channel | $D_{50}$ | $D_{80}$ | $D_{95}$ | $U_{75}$ | $B_{1/3}$ | $B_{2/3}$ |
|---|---|---|---|---|---|---|---|
| Chengdu | Inflow | 0.166667 | 0.583333 | 1.083333 | 1.636364 | 0.279326 | 0.453125 |
| Chengdu | Outflow | 0.166667 | 0.625000 | 1.083333 | 1.636364 | 0.287192 | 0.471179 |
| Shanghai | Inflow | 0.281038 | 0.708333 | 1.096048 | 1.636364 | 0.311065 | 0.489387 |
| Shanghai | Outflow | 0.250000 | 0.674491 | 1.083333 | 1.590909 | 0.311294 | 0.492517 |

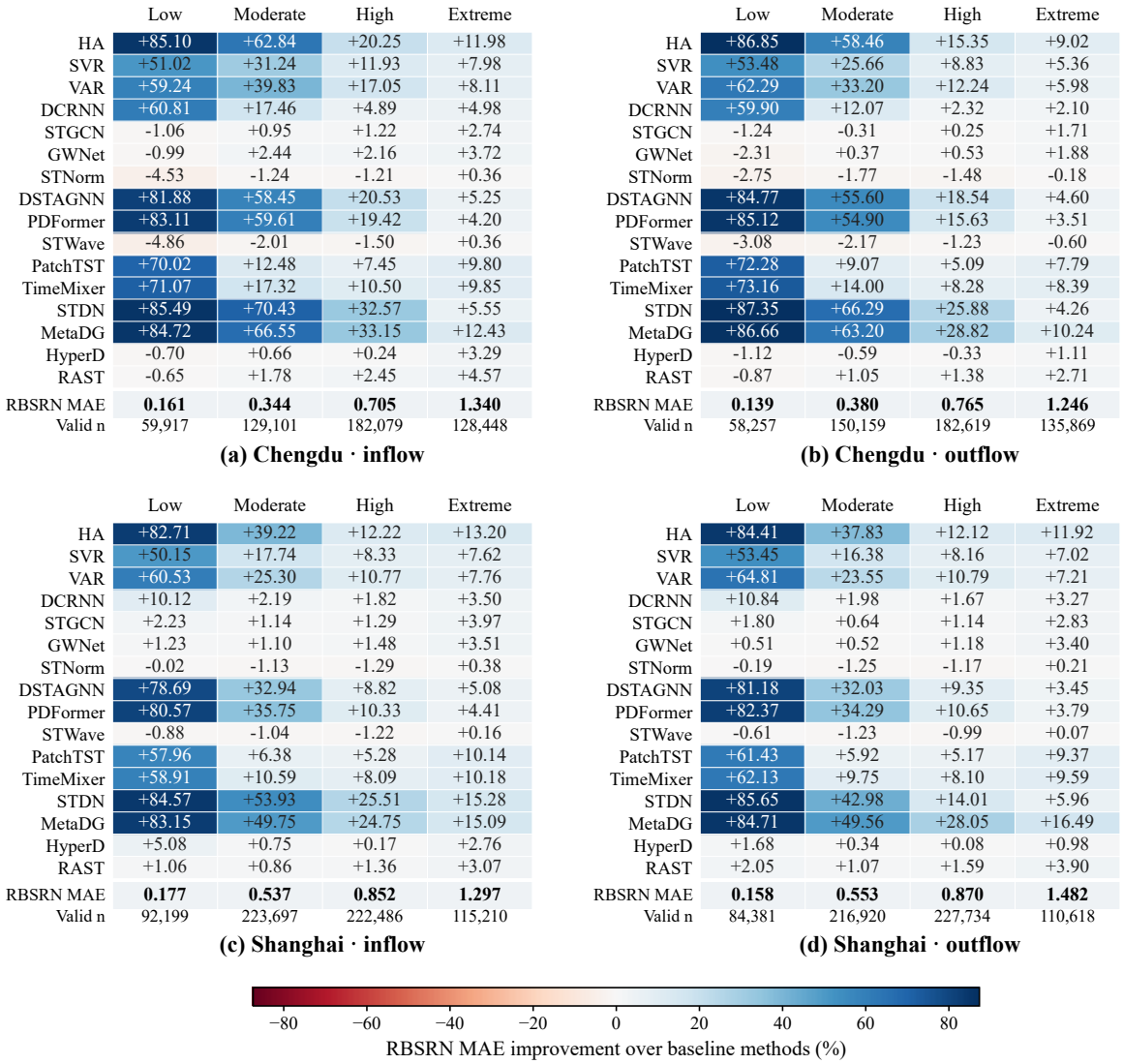



**Figure 6** H6 prediction performance of RBSRN relative to each baseline method across levels of deviation from routine patterns.

Figure 6 compares methods within the same groups; positive values indicate the MAE reduction of RBSRN relative to the corresponding baseline method. In the Extreme group, RBSRN achieved the lowest MAE for Chengdu inflow, Shanghai inflow, and Shanghai outflow.

Its MAE was 0.36%, 0.16%, and 0.07% lower than that of STWave, respectively. For Chengdu outflow, RBSRN ranked third, with MAE 0.60% higher than that of STWave. In the Moderate and High groups, STWave and STNorm had lower errors than RBSRN on all four tasks.

Despite its higher relative ranking in the Extreme group, RBSRN's absolute MAE increased with deviation level on all four tasks. MAE in the Extreme group was 7.33–9.37 times that in the Low group. For Chengdu inflow, for example, MAE increased from 0.161 to 1.340, while mean positive-target flow increased from 1.160 to 3.637 counts per 5 min. Differences in group errors therefore coincided with changes in both deviation level and flow magnitude.

#### 5.2.4 Effects of SERR Residual Corrections across Recent-Change Scenarios

Within the high-deviation subset combining High and Extreme observations ($D > D_{80}$), recent-change intensity is defined by successive differences in historical standardized residuals:

$$U = \frac{1}{11} \sum_{k=2}^{12} |r_k - r_{k-1}|. \tag{26}$$

The 75th percentile of $U$ among high-deviation training samples separates the Slow and Rapid groups; samples above the threshold enter Rapid.

To examine each residual correction in the trained model, we performed a fixed-model residual-removal analysis. Model parameters, gates, and all other outputs were held fixed while a specified residual correction was subtracted from the final prediction:

$$\Delta\text{MAE}_e = \text{MAE}(\widehat{Y} - C_e) - \text{MAE}(\widehat{Y}), \tag{27}$$

Here, $C_e$ is the signed correction actually added to the prediction, including the original gating weight, learnable strength coefficient, and inverse-standardization scale. For the SERR self-evolution expert, removal subtracts the weighted difference between its prediction and the base prediction. For the directed time-lagged expert, removal subtracts the weighted time-lagged correction; BCMS removal subtracts the entire cross-region correction. Gating weights retain their original values and are not renormalized after removal. A positive difference indicates that the original residual correction reduced the group's mean MAE. Each difference measures the net effect on group-average error; the nonlinear absolute loss means that differences for separate corrections cannot be added directly. This diagnostic complements the independently retrained ablations in Section 5.3. It measures the fixed model's sensitivity to existing corrections, whereas the ablations compare correction-pathway configurations after retraining.

In Figure 7, all 16 removal differences for the two experts across eight city–channel–change groups were positive. The removal cost of the self-evolution expert's residual correction in Rapid was 1.15–2.82 times that in Slow. This correction therefore provided larger absolute MAE benefits in the Rapid samples. The directed time-lagged expert's residual correction provided smaller positive contributions. Its removal cost was higher in Rapid than in Slow for Chengdu, but lower for Shanghai.

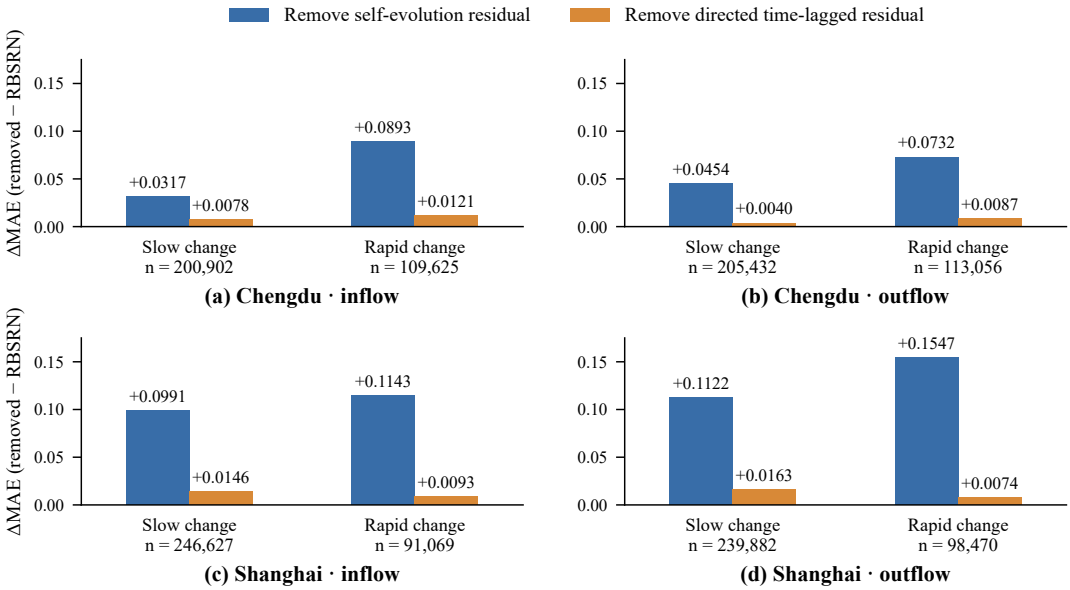



**Figure 7** H6 residual-removal sensitivity of SERR across recent-change groups in high-deviation scenarios.

### 5.2.5 Effects of BCMS Residual Corrections across Background Discrepancies and Forecast Horizons

Similarity between source and target traffic backgrounds is measured by a history-derived proxy for background discrepancy. Let $\mathcal{C}_i$ denote the valid fixed candidate-source set for target grid cell $i$, and $\bar{r}_i$ its mean standardized residual over the 12 historical steps. Within each channel, the proxy is defined as

$$B_i = \frac{1}{|\mathcal{C}_i|} \sum_{j \in \mathcal{C}_i} |\bar{r}_i - \bar{r}_j|. \tag{28}$$

The candidate set follows the model's fixed support. The proxy $B_i$ describes background discrepancy in the observed history and is defined separately from the low-frequency representations learned by BCMS. Within the high-deviation subset, tertile thresholds of $B_i$ from high-deviation training samples define Low, Medium, and High background-discrepancy groups (Table 9). Following Eq. (27), the entire BCMS correction was removed separately at H1–H12 to examine its effects across background conditions and forecast horizons. Both BCMS figures show MAE changes in units of $10^{-3}$ counts per 5 min, so each cell displays $1000 \times \Delta$MAE.

Figure 8 shows that, for every city, channel, and forecast horizon, the mean correction benefit was higher in the Low group than in the Medium group. The Medium group, in turn, had a higher benefit than the High group. Of the 144 background-group–horizon cells, 143 were positive. For Chengdu outflow, the High group at H5 had the sole negative value, approximately $-4.8 \times 10^{-6}$. Benefits fluctuated across horizons within each group, without a monotonic increase with forecast horizon.

Figure 9 extends the analysis to the full test set, retaining the background-group thresholds fitted using high-deviation training histories. Removal differences were positive in all 144 background-group–horizon cells and exceeded the corresponding values for the high-deviation subset alone. The ordering persisted: benefits were higher in the Low group than in the Medium

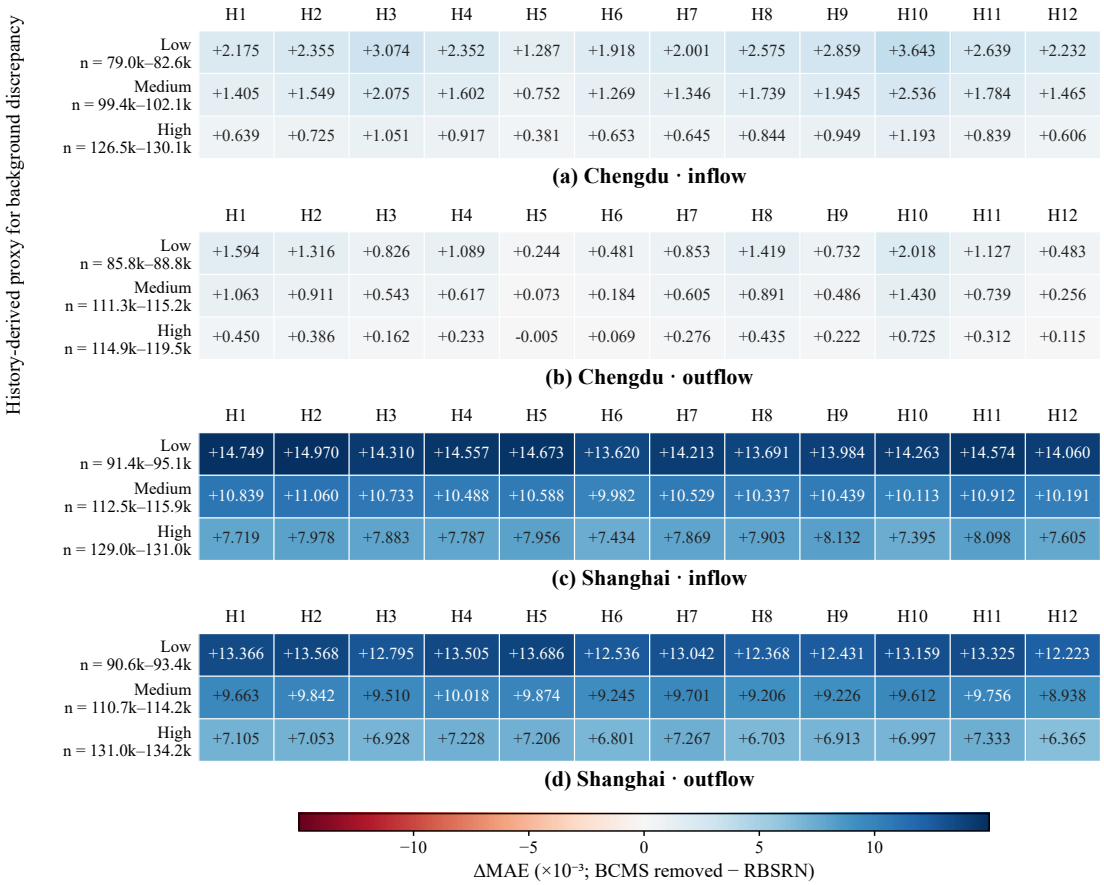



**Figure 8** Sensitivity of BCMS to background discrepancy and forecast horizon in high-deviation scenarios.

group, and higher in the Medium group than in the High group. For Shanghai inflow at H6, the full-test-set differences in the three background groups were 0.025547, 0.015634, and 0.009873, respectively. The corresponding values for the high-deviation subset were 0.013620, 0.009982, and 0.007434. The two analysis scopes are nested; the full-test-set results indicate that the average benefit of BCMS corrections extends across a broader range of traffic conditions.

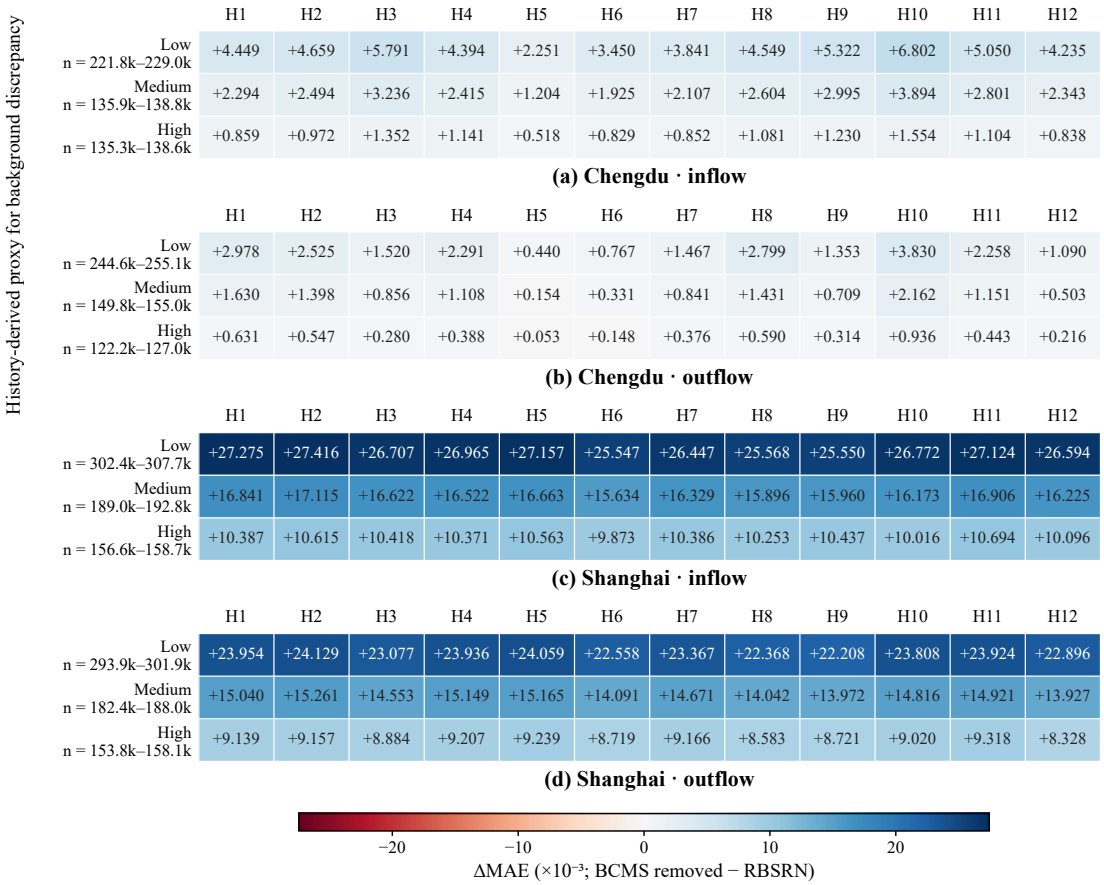



**Figure 9** Sensitivity of BCMS to background discrepancy and forecast horizon across the full test set.

## 5.3 Ablation Study

We evaluated the contribution of each residual correction branch by comparing the full RBSRN with three ablation variants on Chengdu. The variants remove the self-evolution expert (A1), the directed time-lagged expert (A2), or BCMS (A3). Each variant was retrained independently for a maximum of 200 epochs, with an early-stopping patience of 20 epochs. In A1 and A2, the removed expert's gating weight was set to zero, and the remaining expert weights were normalized to sum to 1. Testing followed the positive-target mask and AVG12 aggregation defined in Section 5.1.2. Joint MAE and joint RMSE are the equal-weight averages of the corresponding inflow and outflow AVG12 metrics.

**Table 10** Per-channel and joint AVG12 metrics for the Chengdu ablation study

| Model configuration | Inflow MAE | Inflow RMSE | Outflow MAE | Outflow RMSE | Joint MAE | Joint RMSE |
|---|---|---|---|---|---|---|
| RBSRN | **0.7084** | **1.2935** | **0.7087** | **1.2235** | **0.7085** | **1.2585** |
| w/o Self-evolution expert (A1) | 0.7129 | 1.3037 | 0.7121 | 1.2270 | 0.7125 | 1.2653 |
| w/o Directed time-lagged expert (A2) | 0.7105 | 1.3048 | 0.7099 | 1.2264 | 0.7102 | 1.2656 |
| w/o BCMS (A3) | 0.7109 | 1.3014 | 0.7102 | 1.2276 | 0.7105 | 1.2645 |

Table 10 shows that the full RBSRN achieved the lowest error on all six metrics. Removing any branch increased MAE and RMSE for both inflow and outflow. Removing the self-evolution expert increased joint MAE by 0.56%, the largest increase among the three variants. Removing the directed time-lagged expert increased joint RMSE by 0.56%, slightly exceeding the 0.54%

increase from removing the self-evolution expert. These results indicate that both expert types contributed to reducing prediction error, with self-evolution correction having a larger effect on joint MAE.

Removing BCMS increased joint MAE and joint RMSE by 0.28% and 0.48%, respectively, with higher errors for both inflow and outflow. This suggests that background-conditioned historical messages complement the role-specific residual correction provided by SERR. The full model achieved the lowest error when all three branches were present. These results support combining node self-evolution, directed time-lagged association, and background-conditioned message selection to refine predictions.

## 5.4 Hyperparameter Sensitivity Analysis

We conducted one-factor-at-a-time validation experiments on Chengdu to examine how SERR gating capacity, BCMS representation capacity, and the initial values of two residual pathways affect prediction error. The SERR gating hidden dimension and the BCMS internal representation dimension each took values in $\{8, 16, 32, 64, 128\}$. Both $\alpha_0$ and $\beta_0$ took values in $\{0, 0.001, 0.01, 0.05, 0.1\}$. They specify the initial values of the learnable residual parameters defined in Sections 4.3 and 4.4, respectively, and these parameters continue to update during training. The reference configuration was $(32, 32, 0, 0)$, as used in the main comparison experiments. We changed one parameter at a time while keeping all other settings fixed. Bold tick labels indicate the reference configuration, and the minimum within each group was determined from unrounded results. Training, checkpoint selection, and validation aggregation followed Sections 5.1.3 and 5.1.1.

We first constructed the model with the reference configuration and replaced the gating or BCMS module only when its dimension differed from 32. Residual initial values were assigned directly to the learnable parameters. Neither training nor validation used additional splitting into microbatches.

All four parameters produced only small changes in joint validation error within the tested ranges (Figure 10). We calculated each group's relative range as the difference between its maximum and minimum errors divided by the error of the reference configuration. Across the four groups, this range was below 0.15% for joint MAE and below 0.35% for joint RMSE.

Increasing the hidden dimension did not consistently reduce error, and the lowest joint MAE and RMSE occurred at different dimensions. The best positive initial values of the residual parameters provided only small improvements over zero initialization. The two metrics likewise did not share a single optimal initial value.

Figure 11 presents the complete results for each flow channel. The small changes in joint error also included different responses across channels. For the SERR gating dimension, inflow MAE was lowest at 64, whereas outflow MAE was lowest at 8. Thus, the two flow channels responded differently to gating capacity.

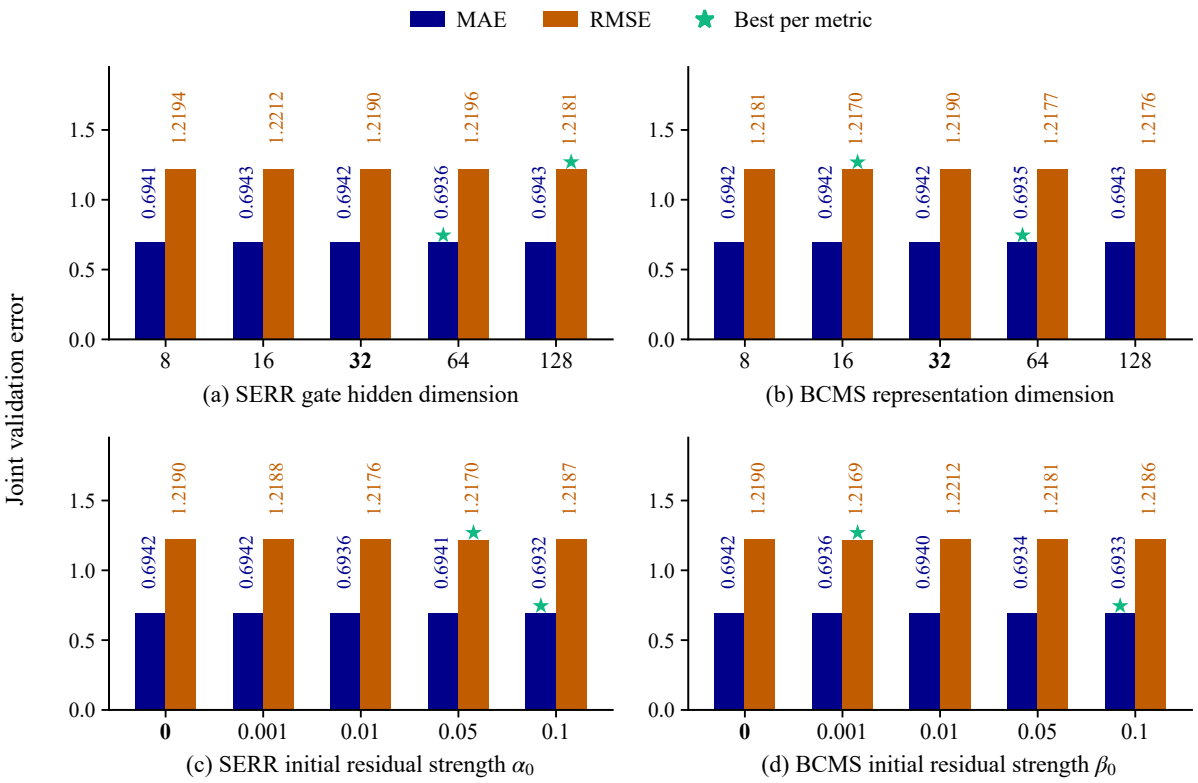



**Figure 10** Hyperparameter sensitivity of joint validation metrics on the Chengdu dataset.

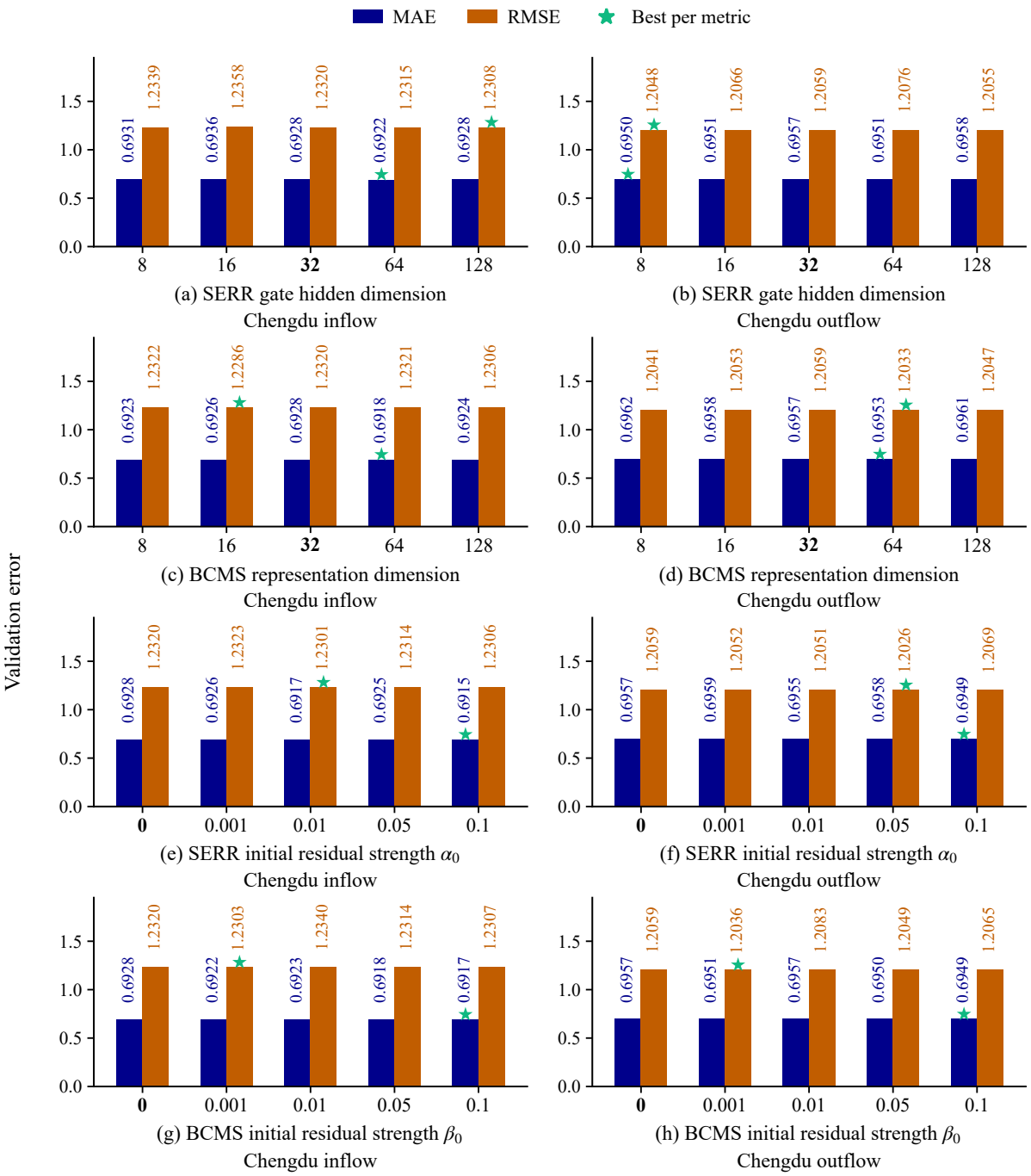



**Figure 11** Hyperparameter sensitivity for inflow and outflow on the Chengdu dataset.

## 5.5 Model Size and Inference Efficiency

With both the gating and BCMS dimensions set to 32, RBSRN contains approximately 7.071 million trainable parameters. We evaluated inference efficiency using the Chengdu checkpoint from the main comparison experiments. Each batch-size setting used a single NVIDIA A100-SXM4-40GB GPU with FP32 tensors; the remaining computing environment is described in Section 5.1.3. Each window supplies 12 historical flow steps and jointly produces inflow and outflow forecasts for 1,024 nodes over the next 12 steps.

For batch size 1, we used compiled BCMS operators and CUDA Graph, with 20 warm-up iterations followed by 100 timed iterations. For batch size 64, we used compiled BCMS operators, with 10 warm-up iterations followed by 30 timed iterations. CUDA synchronization was performed before and after each measurement. We report the median and 95th percentile (P95), which describe typical execution time and tail latency within these measurements, respectively.

Forward-only timing includes model computation on GPU-resident inputs and excludes copying new inputs. End-to-end timing covers memory-resident data access, wavelet decomposition, normalization, state-evidence construction, CPU/GPU transfers, model inference, inverse standardization, and returning results to the CPU. It also includes input updates before CUDA Graph replay. Both measurements exclude startup preparation, offline graph construction, and cold disk reads. Startup preparation comprises initialization, compilation, graph capture, and warm-up.

**Table 11** RBSRN inference time on the Chengdu dataset

| Inference setting | Batch size | Forward only ms/batch | | End to end ms/batch | |
|---|---|---|---|---|---|
| | | Median | P95 | Median | P95 |
| Single-window forecasting | 1 | 15.907 | 15.930 | 19.228 | 19.507 |
| Batch forecasting | 64 | 1138.819 | 1310.517 | 1280.394 | 1464.308 |

Table 11 reports median forward-only and end-to-end latencies of 15.907 ms and 19.228 ms for single-window forecasting, with an end-to-end P95 of 19.507 ms. In this test environment, the measured computation time fit within a 5-minute update cycle. At batch size 64, the median end-to-end time was 1,280.394 ms/batch, and the amortized forward-only time was 17.794 ms/window. The batch-size-1 measurements characterize the response time of an individual request.

The inference configurations above used the same model checkpoint. We compared each against the reference implementation at the same batch size. At batch size 64, checking all 1,723 test windows yielded a maximum absolute prediction difference of $7.63 \times 10^{-6}$. Batch size 1 passed a sampled check against the reference at the same batch size. Aggregate forecasting metrics were similar across the two batch sizes, although individual predictions were not identical. All main accuracy results were evaluated at batch size 64.

# 6 Conclusion

We present RBSRN, a role-aware and background-conditioned spatiotemporal residual network for short-term, multi-step regional inflow and outflow forecasting. It refines a shared base prediction through state-conditioned expert correction and background-conditioned historical message selection. In experiments over forecast horizons of 5–60 minutes in Chengdu and Shanghai, we compared RBSRN with 13 baseline methods under positive-target evaluation. RBSRN achieved the lowest AVG12 MAE and RMSE across all four city–channel tasks. Its MAE was 0.43%–2.00% lower than that of the strongest baseline method for each task.

Ablation experiments and fixed-model diagnostics supported complementary roles for SERR and BCMS. In H6 diagnostics on high-deviation samples, self-evolution correction provided larger absolute MAE benefits for rapidly changing histories than for slowly changing histories. The effects of directed time-lagged correction varied across cities. BCMS provided larger mean benefits when the history-derived proxy for background discrepancy was smaller, and these benefits extended beyond high-deviation scenarios. RBSRN did not outperform the strongest baseline methods in every deviation group, indicating that its advantages depended on the traffic state.

The empirical evaluation is limited to one month of data from each city. Future work should evaluate temporal generalization across months and seasons, assess the benefits and computational costs of dynamically updating candidate relations, and compare all methods under both positive-target and all-grid evaluation. These extensions would clarify the robustness and practical applicability of the proposed correction mechanisms.

# References


[1] F. Rodrigues, I. Markou, and F. C. Pereira, "Combining time-series and textual data for taxi demand prediction in event areas: A deep learning approach," *Inf. Fusion*, vol. 49, pp. 120–129, 2019, doi: 10.1016/j.inffus.2018.07.007.

[2] J. Sun, J. Zhang, Q. Li, X. Yi, Y. Liang, and Y. Zheng, "Predicting citywide crowd flows in irregular regions using multi-view graph convolutional networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2348–2359, 2022, doi: 10.1109/TKDE.2020.3008774.

[3] E. Kontou, V. Garikapati, and Y. Hou, "Reducing ridesourcing empty vehicle travel with future travel demand prediction," *Transp. Res. C, Emerg. Technol.*, vol. 121, Art. no. 102826, 2020, doi: 10.1016/j.trc.2020.102826.

[4] J. Ke, H. Zheng, H. Yang, and X. Chen, "Short-term forecasting of passenger demand under on-demand ride services: A spatio-temporal deep learning approach," *Transp. Res. C, Emerg. Technol.*, vol. 85, pp. 591–608, 2017, doi: 10.1016/j.trc.2017.10.016.

[5] A. Essien, I. Petrounias, P. Sampaio, and S. Sampaio, "A deep-learning model for urban traffic flow prediction with traffic events mined from Twitter," *World Wide Web*, vol. 24, no. 4, pp. 1345–1368, 2021, doi: 10.1007/s11280-020-00800-3.

[6] J. Zhang, Y. Zheng, and D. Qi, "Deep spatio-temporal residual networks for citywide crowd flows prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, 2017, pp. 1655–1661, doi: 10.1609/aaai.v31i1.10735.

[7] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.

[8] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3634–3640, doi: 10.24963/ijcai.2018/505.

[9] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, "Graph WaveNet for deep spatial-temporal graph modeling," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2019, pp. 1907–1913, doi: 10.24963/ijcai.2019/264.

[10] Y. Fang *et al.*, "When spatio-temporal meet wavelets: Disentangled traffic forecasting via efficient spectral graph attention networks," in *Proc. IEEE 39th Int. Conf. Data Eng. (ICDE)*, 2023, pp. 517–529, doi: 10.1109/ICDE55515.2023.00046.

[11] Y. Xu, L. Wei, and L. Feng, "A spatial-temporal trend-event decoupling dual-channel framework for traffic flow prediction," *Expert Syst. Appl.*, vol. 287, Art. no. 128107, 2025, doi: 10.1016/j.eswa.2025.128107.

[12] X. Zhang *et al.*, "Traffic flow forecasting with spatial-temporal graph diffusion network," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 17, 2021, pp. 15008–15015, doi: 10.1609/aaai.v35i17.17761.

[13] Q. Long, Z. Fang, C. Fang, C. Chen, P. Wang, and Y. Zhou, "Unveiling delay effects in traffic forecasting: A perspective from spatial-temporal delay differential equations," in *Proc. ACM Web Conf.*, 2024, pp. 1035–1044, doi: 10.1145/3589334.3645688.

[14] Z. Shao *et al.*, "Decoupled dynamic spatial-temporal graph neural network for traffic forecasting," *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 2733–2746, 2022, doi: 10.14778/3551793.3551827.

[15] H. Lee and S. Ko, "TESTAM: A time-enhanced spatio-temporal attention model with mixture of experts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[16] J. Jiang, C. Han, W. X. Zhao, and J. Wang, "PDFormer: Propagation delay-aware dynamic long-range Transformer for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 4, 2023, pp. 4365–4373, doi: 10.1609/aaai.v37i4.25556.

[17] R. Jiang *et al.*, "DL-Traff: Survey and benchmark of deep learning models for urban traffic prediction," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2021, pp. 4515–4525, doi: 10.1145/3459637.3482000.

[18] Z. Shao, Z. Zhang, F. Wang, W. Wei, and Y. Xu, "Spatial-temporal identity: A simple yet effective baseline for multivariate time series forecasting," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, 2022, pp. 4454–4458, doi: 10.1145/3511808.3557702.

[19] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 922–929, doi: 10.1609/aaai.v33i01.3301922.

[20] C. Zheng, X. Fan, C. Wang, and J. Qi, "GMAN: A graph multi-attention network for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 1, 2020, pp. 1234–1241, doi: 10.1609/aaai.v34i01.5477.

[21] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2020, pp. 753–763, doi: 10.1145/3394486.3403118.

[22] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, "Adaptive graph convolutional recurrent network for traffic forecasting," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 17804–17815.

[23] C. Song, Y. Lin, S. Guo, and H. Wan, "Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 1, 2020, pp. 914–921, doi: 10.1609/aaai.v34i01.5438.

[24] M. Li and Z. Zhu, "Spatial-temporal fusion graph neural networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 5, 2021, pp. 4189–4196, doi: 10.1609/aaai.v35i5.16542.

[25] X. Su, F. Liu, Y. Chang, E. Tanin, M. Sarvi, and J. Qi, "DualCast: A model to disentangle aperiodic events from traffic series," in *Proc. 34th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2025, pp. 3290–3298, doi: 10.24963/ijcai.2025/366.

[26] M. Shao, Z. Zhang, Y. Wang, Y. Dai, X. Shen, and X. Wang, "HyperD: Hybrid periodicity decoupling framework for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 18, 2026, pp. 15689–15697, doi: 10.1609/aaai.v40i18.38599.

[27] J. Ji *et al.*, "Spatio-temporal self-supervised learning for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 4, 2023, pp. 4356–4364, doi: 10.1609/aaai.v37i4.25555.

[28] M. Li, P. Tong, M. Li, Z. Jin, J. Huang, and X.-S. Hua, "Traffic flow prediction with vehicle trajectories," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 1, 2021, pp. 294–302, doi: 10.1609/aaai.v35i1.16104.

[29] H. Yao, X. Tang, H. Wei, G. Zheng, and Z. Li, "Revisiting spatial-temporal similarity: A deep learning framework for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, no. 1, 2019, pp. 5668–5675, doi: 10.1609/aaai.v33i01.33015668.

[30] R. Jiang *et al.*, "Spatio-temporal meta-graph learning for traffic forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 7, 2023, pp. 8078–8086, doi: 10.1609/aaai.v37i7.25976.

[31] W. Ruan, X. Dang, Z. Zhou, S. Lyu, and Y. Liang, "A retrieval augmented spatio-temporal framework for traffic prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 46, 2026, pp. 39163–39172, doi: 10.1609/aaai.v40i46.41264.

[32] Z. Pan *et al.*, "Spatio-temporal meta learning for urban traffic prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1462–1476, 2022, doi: 10.1109/TKDE.2020.2995855.

[33] J. Ke, X. Qin, H. Yang, Z. Zheng, Z. Zhu, and J. Ye, "Predicting origin-destination ride-sourcing demand with a spatio-temporal encoder-decoder residual multi-graph convolutional network," *Transp. Res. C, Emerg. Technol.*, vol. 122, Art. no. 102858, 2021, doi: 10.1016/j.trc.2020.102858.

[34] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 3942–3951, doi: 10.1609/aaai.v32i1.11671.

[35] S. Lan, Y. Ma, W. Huang, W. Wang, H. Yang, and P. Li, "DSTAGNN: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, vol. 162, 2022, pp. 11906–11917.

[36] L. Cao, B. Wang, G. Jiang, Y. Yu, and J. Dong, "Spatiotemporal-aware trend-seasonality decomposition network for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 11, 2025, pp. 11463–11471, doi: 10.1609/aaai.v39i11.33247.

[37] Y. Zou, H. Yuan, Q. Yang, Z. Yuan, S. Wang, and S. Ruan, "Meta dynamic graph for traffic flow prediction," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 19, 2026, pp. 16584–16592, doi: 10.1609/aaai.v40i19.38699.

[38] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with Transformers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[39] S. Wang *et al.*, "TimeMixer: Decomposable multiscale mixing for time series forecasting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[40] J. Deng, X. Chen, R. Jiang, X. Song, and I. W. Tsang, "ST-Norm: Spatial and temporal normalization for multi-variate time series forecasting," in *Proc. 27th ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD)*, 2021, pp. 269–278, doi: 10.1145/3447548.3467330.